SpiralTrain
developer courses

Course Schedule

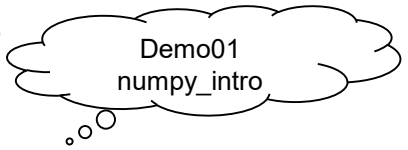
- **Numpy**
- Pandas Intro
- Extracting and Transforming Data
- Advanced Indexing
- Rearranging Data
- Grouping Data
- MatplotLib
- Time Series

- **What is Numpy?**
- **NumPy Speed**
- **Python Integers**
- **Python versus C Integer**
- **Numpy Array versus Python List**
- **Vectorization and Broadcasting**
- **Basic Operations**
- **Numpy Data Types**
- **nparray**
- **Multidimensional Array**
- **Axis and Dimensions**
- **Numpy Functions**
- **Reshape and Transpose**
- **Memory Layouts**
- **C and F order**
- **Indexing and Slicing**
- **Ones and Zeros**
- **Array Copy**
- **Identity and Eye Function**
- **Ravel and Stack**

© copyright : spiraltrain@gmail.com

What is Numpy?

- **Numpy is Python module for scientific computing :**
  - Python extension written in C
  - Used for fast vector and matrix operations
- **Numpy contains among other things :**
  - Powerful N-dimensional array object
  - Sophisticated broadcasting functions
  - Tools for integrating C/C++ and Fortran code
  - Useful linear algebra, Fourier transform and random number capabilities
- **Core of NumPy package is `ndarray` object :**
  - Encapsulates n-dimensional arrays of homogeneous data types
  - Many operations performed in compiled code for performance
- **Differences NumPy arrays and standard Python sequences :**
  - NumPy arrays have fixed size at creation, Python lists which grow dynamically
  - Elements in NumPy array are all of same data type, same size
  - NumPy arrays allow mathematical of operations on large data



NumPy Speed

- **Sequence size and speed are important in scientific computing**
- **Mutiplying elements of two sequences in Python :**
  - Gives correct answer but you pay price of inefficiency of looping in Python

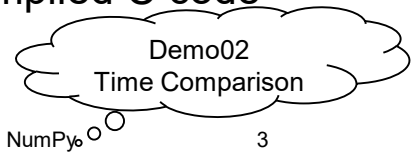
```
c = []
for i in range(len(a)):
    c.append(a[i]*b[i])
```

- **Same task is done with more speed in C :**
  - Saves overhead in interpreting Python code and manipulating Python objects
  - At the expense of the benefits gained from coding in Python

```
for (i = 0; i < rows; i++): {
  c[i] = a[i]*b[i];
}
```

- **NumPy gives us best of both worlds :**
  - Element-by-element operations are “default mode” when an `ndarray` is involved
  - Element-by-element operation is speedily executed by pre-compiled C code

```
c = a * b
```



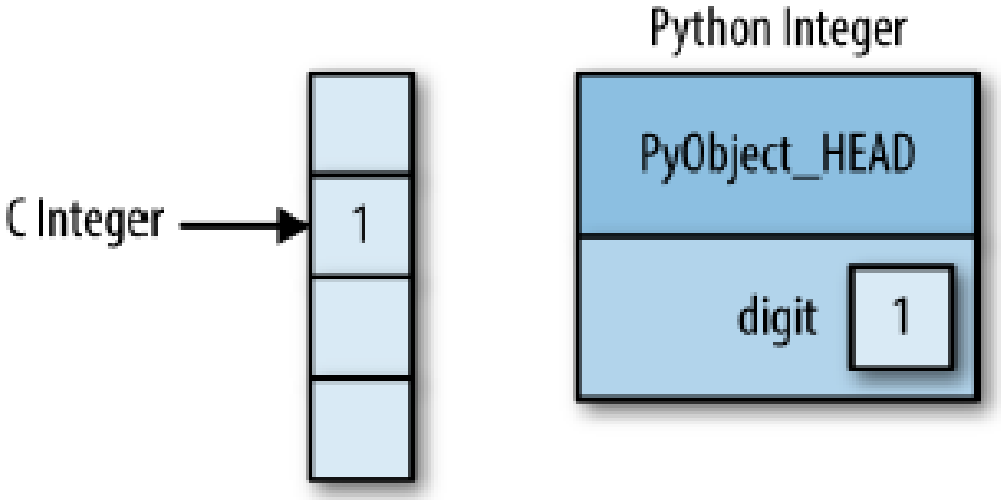
Python Integers

- **Standard Python implementation is written in C :**
  - Every Python object is simply a cleverly disguised C structure
- **Integer in Python is not just a “raw” integer :**
  - It’s actually a pointer to a compound C structure

```
struct _longobject {
    long ob_refcnt;
    PyTypeObject *ob_type;
    size_t ob_size;
    long ob_digit[1];
};
```

- **Single integer in Python actually contains four pieces :**
  - `ob_refcnt` reference count to silently handle memory allocation and deallocation
  - `ob_type` encodes the type of the variable
  - `ob_size` specifies the size of the following data members
  - `ob_digit` contains integer value that we expect Python variable to represent

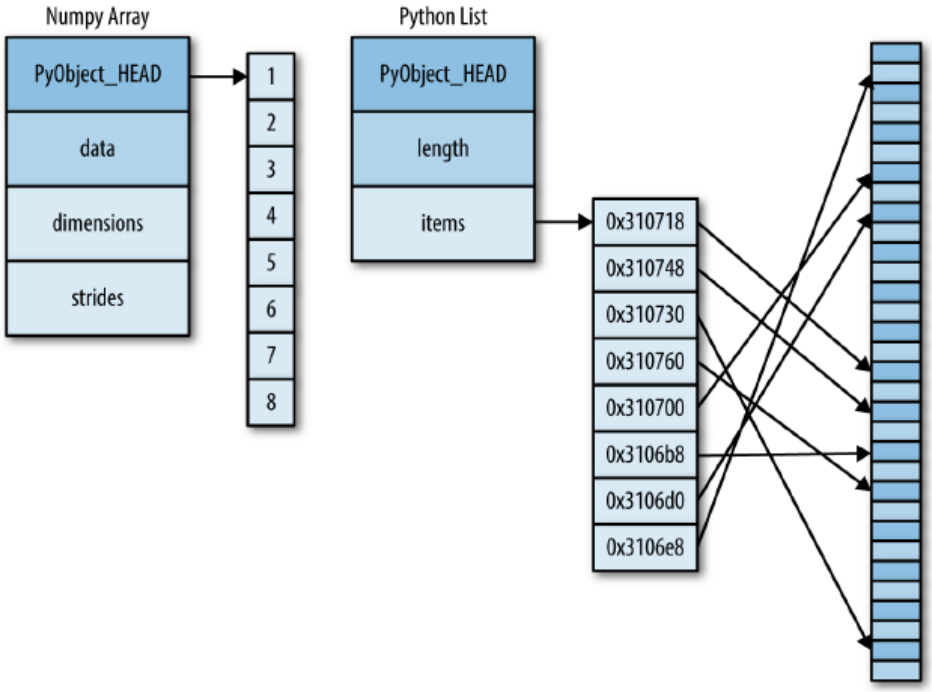
Python versus C Integers



- `PyObject_HEAD` :
  - Part of structure containing reference count, type code, etc
- **C integer :**
  - Label for position in memory whose bytes encode an integer value
- **Python integer :**
  - Pointer to position in memory containing all Python object information

Numpy Array versus Python List

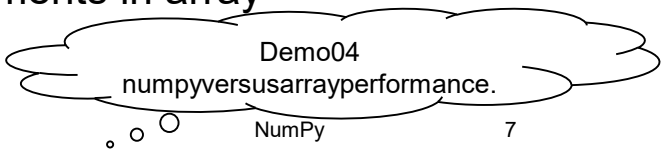
- **Numpy array :**
  - Essentially contains single pointer to one contiguous block of data
- **Python list :**
  - Contains pointer to block of pointers, each of which points to full Python object



Demo03
numpyversusarraysize

Vectorization and Broadcasting

- **Vectorization is absence of explicit looping, indexing in code :**
  - Takes place behind the scenes in optimized, pre-compiled C code
- **Benefits of vectorized code :**
  - Vectorized code is more concise and easier to read
  - Fewer lines of code generally means fewer bugs
  - Code more closely resembles standard mathematical notation
  - Vectorization results in more “Pythonic” code
- **Broadcasting refers to element-by-element behavior of operation :**
  - In NumPy all operations behave in this implicit element-by-element fashion
  - Works for different type also for multidimensional arrays and scalar and array
- **NumPy fully supports an object-oriented approach :**
  - `ndarray` is a class, possessing numerous methods and attributes
  - `ndarray.ndim` : number of axes or dimensions of the array
  - `ndarray.dtype` : object describing type of elements in array



Basic Operations

- **Arithmetic operators on arrays apply elementwise :**
  - New array is created and filled with the result

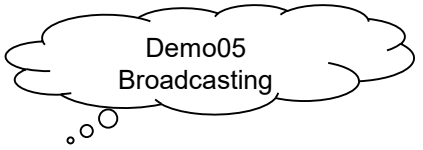
```
a = np.array([20,30,40,50])
b = np.arange(4)
print (b) # [0, 1, 2, 3])
c = a-b
print (c) # [20, 29,38, 47])
```

- **Elementwise operation done for operators and functions :**

```
 b**2
print (b)              # [0, 1, 4, 9]
print (10*np.sin(a)) # [ 9.12945, -9.88031, 7.4511 , -2.62374]
```

- **Product operator * operates elementwise in NumPy arrays :**
  - Matrix product can be performed using the dot function or method

```
A = np.array( [[1,1], [0,1]] )
B = np.array( [[2,0], [3,4]] )
print (A*B) # elementwise product : [2, 0],[0, 4]]
print (A.dot(B)) # matrix product [[5, 4],[3, 4]])
```



Numpy Data Types

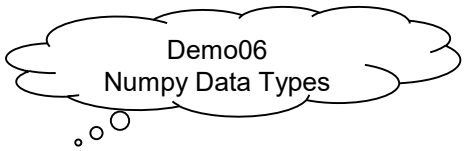
- **Data type object `dtype` is an instance of `numpy.dtype` class :**
  - Can be created with `numpy.dtype`

```
i16 = np.dtype(np.int16)
print(i16)
```

```
lst = [ [3.4, 8.7, 9.9],
        [1.1, -7.8, -0.7],
        [4.1, 12.3, 4.8] ]
A = np.array(lst, dtype=i16)
print(A)
```

- **Can expect following result, if we execute code above :**

```
int16
[[ 3   8   9]
 [ 1  -7   0]
 [ 4  12   4]]
```



- **Other Data types in NumPy are :**
  - `int8, int32, int64, float16, float32, float64, object`

nparray

- **NumPy's `nparray` object is homogeneous multidimensional array :**
  - Dimensions are called axes, number of axes is rank
- **In following code array has rank 2 :**
  - First dimension has length of 2, second dimension has length of 3

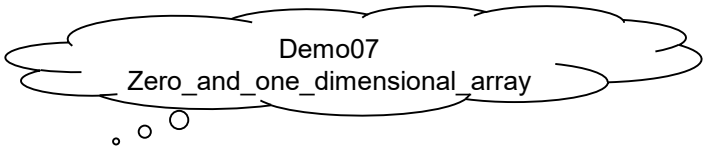
```
[[ 1, 0, 0], [ 0, 1, 2]]
```

- **Zero dimensional array is created as follows :**

```
x = np.array(42)
print(type(x))      #<class 'numpy.ndarray'>
print(np.ndim(x))   #0
```

- **One dimensional array is created as follows :**

```
F = np.array([1, 1, 2, 3, 5, 8, 13, 21])
V = np.array([3.4, 6.9, 99.8, 12.8])
print(F.dtype)      # int64
print(V.dtype)      # float64
print(np.ndim(F))   1
print(np.ndim(V))   1
```



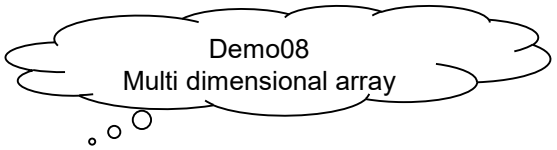
Multidimensional Array

- `ndarray.shape` :
  - Tuple of integers indicating size of array in each dimension
  - For a matrix with n rows and m columns, shape will be (n,m)

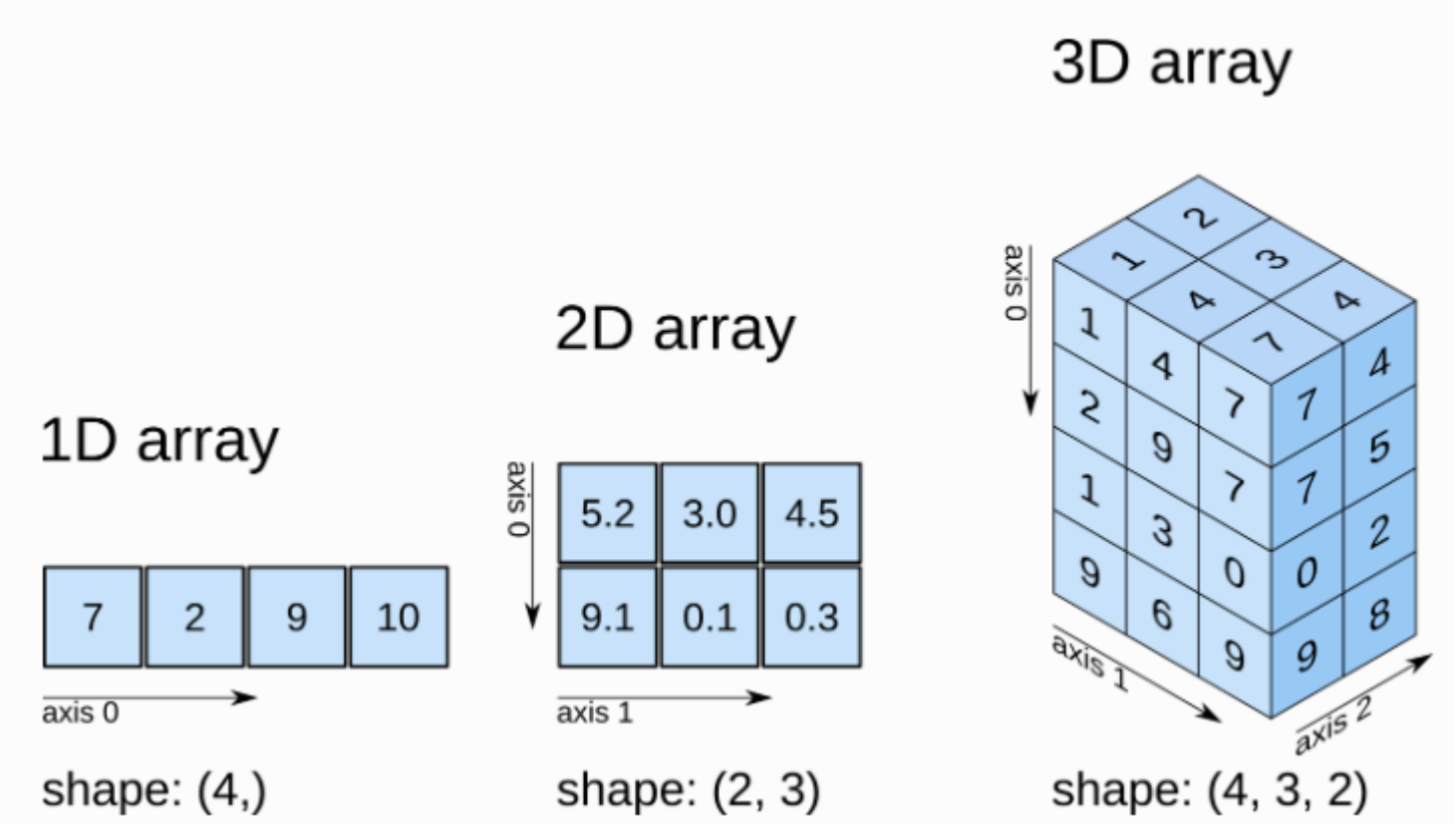
```
A = np.array([ [3.4, 8.7, 9.9],
               [1.1, -7.8, -0.7],
               [4.1, 12.3, 4.8]])
print(A)
print(A.ndim)    # 2
print(A.shape)   # (3, 3)
```

- **Indices of shape** :
  - First index or dimension indicates number of rows and is indicated with axis 0
  - Second index or dimension indicates number of columns and is indicated with axis 1

```
B = np.array([[[111, 112], [121, 122]],
              [[211, 212], [221, 222]],
              [[311, 312], [321, 322]]])
print(B)
print(B.ndim)    # 3
print(B.shape)   # (3, 2, 2)
```



Axis and Dimensions



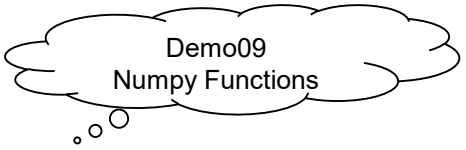
Numpy Functions

```
a = np.array([1, 2, 3])
print(a.min())  # 1
print(a.max())  # 3
print(a.sum())  # 6

a= np.array([(1,2,3),(3,4,5)])
print(a.sum(axis=0))  # [4 6 8]
print(a.sum(axis=1))  # [6 12]

a=np.array([(1,2,3),(3,4,5,)])
print(np.sqrt(a))
print(np.std(a))

# divide difference 1 3 in 10 equal parts
a=np.linspace(1,3,10)
print(a)
```



Reshape and Transpose

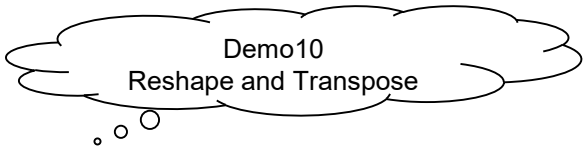
- `numpy.reshape` **shapes an array without changing data of array :**
  - Can only reshape if total number of elements remain the same
- `numpy.transpose` **reverse the dimensions of the array :**
  - Make rows into columns and colums into row for two dimensional array
  - Returns a `ndarrray` with its axes permuted. View is returned whenever possible
  - Can also be achieved by accessing T attribute

```
import numpy as np
a = np.array([(8,9,10),(11,12,13)])
print(a)
print(a.shape)

b = a.transpose()   # or a.T
print(b)
print(b.shape)

a=a.reshape(3,2)
print(a)
print(a.shape)

a=a.reshape(2,3)
print(a)
print(a.shape)
```



Iterating Numpy Arrays

- **Iterating over a Numpy array can be done using `for` :**

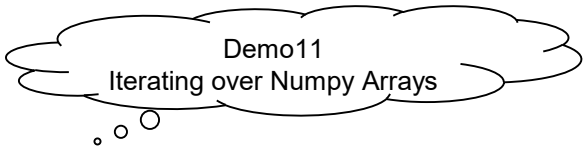
```
ca = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
for i in range(len(ca)):
    for j in range(len(ca[i])):
        print(ca[i][j], end= ' ')
```

- **Better alternative is using an iterator object `numpy.nditer` :**
  - Order of iteration is chosen to match the memory layout of an array

```
a = np.arange(12)
a = a.reshape(3, 4)

print()
print('Original array is:')
print(a)
print()

# iterating  an array
for x in np.nditer(a):
    print(x, end=' '
```

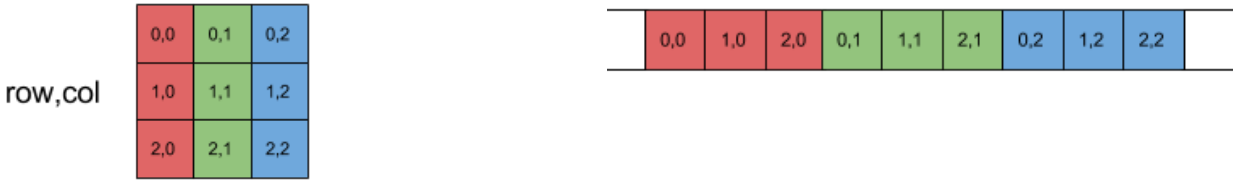


Memory Layout Multi Dimensional Arrays

- **Computer memory is a linear one-dimensional structure :**
  - Mapping multi-dimensional data on memory can be done in several ways
- **Common memory layouts for multi-dimensional array data :**
  - Row-major puts first row in contiguous memory, then second row right after it, etc
  - Column-major layout puts first column in contiguous memory, then the second, etc.
- **2D row major is used in C and is the default in Numpy :**
  - Column indices change the fastest



- **2D column major is used in Fortan and can be used in Numpy :**
  - Row indices change the fastest



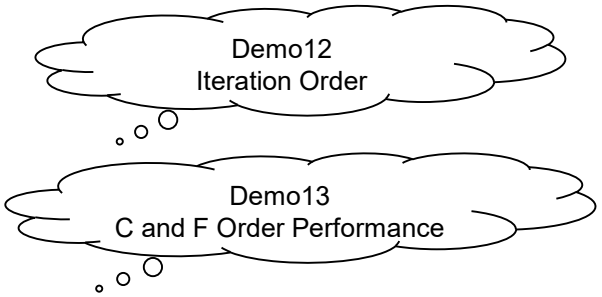
C and F Order

- **Memory layout Numpy arrays can be specified at creation time :**
  - C indicates row major, F indicates columns major, default is C

```
c = np.array(a, order='C')
f = np.array(a, order='F')
```

- **Sometimes important to visit elements of array in specific order :**
  - Irrespective of the layout of the elements in memory.
- `nditer` **object provides order parameter to control order iteration :**
  - Default, order='K' to keep the existing order.
  - Can be overridden with order='C' for C order and order='F' for Fortran order

```
print('Iteration an array in C-style order:')
for x in np.nditer(a, order='C'):
    print(x, end=' ')
print('Iteration an array in F-style order:')
for x in np.nditer(a, order='F'):
    print(x, end=' ')
```



- **Way array is traversed has implications for performance :**
  - Performance is better if subsequent accesses are near each other

Indexing and Slicing

- **One-dimensional arrays can be indexed, sliced and iterated over :**
  - Much like lists and other Python sequences

```
F = np.array([1, 1, 2, 3, 5, 8, 13, 21])
```

- **Print first element of F, i.e. the element with the index 0 :**

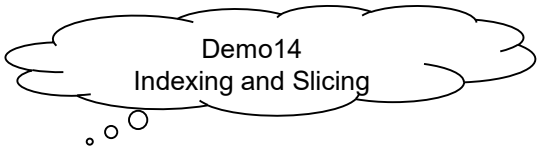
```
print(F[0])       # 1
```

- **Print last element of F :**

```
print(F[-1])      # 21
```

- **Specific location specified by referring to indexes in square braces :**

```
B = np.array([ [[111, 112], [121, 122]],
               [[211, 212], [221, 222]],
               [[311, 312], [321, 322]] ])
print(B[0][1][0]) # 121
```



Ones and Zeros

- **Method `ones(t)` takes tuple `t` :**
  - With shape of the array and fills the array accordingly with ones

```
E = np.ones((2,3))
print(E)
```

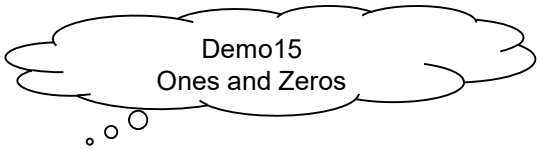
- **Optionally we can specify a data type :**

```
F = np.ones((3,4),dtype=int)
print(F)
```

- **Method `zeros` does the same but now with zeros :**

```
Z = np.zeros((2,4))
print(Z)
```

- **There is also the method `empty` :**
  - Creates an array whose initial content is random
  - Depends on the state of the memory



Array Copy

- `copy` **method makes complete copy of array and its data :**

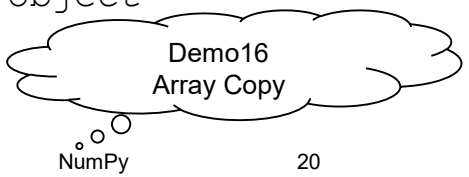
```
x = np.array([[42,22,12],[44,53,66]], order='F')
print(x.flags['C_CONTIGUOUS'])  # False
y = x.copy()
x[0,0] = 1001
print(x)
print(y)
```

- **Copy of original** `F` **order array is in** `C` **order again :**

```
print(y.flags['C_CONTIGUOUS'])  # True
```

- **Changes in original array do not propagate to copy**
- **Simple assignments make no copy of array objects or data :**

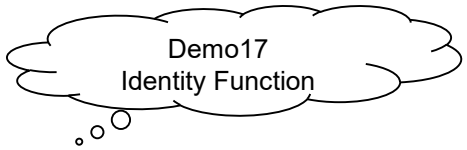
```
a = np.arange(12)
b = a   # no new object is created
b is a  # a and b are two names for the same ndarray object
```



Identity and Eye Function

- `identity` **functions returns identity array :**
  - Square array with ones on the main diagonal

```
np.identity(4)
```



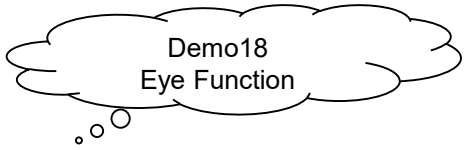
- **Python code above returns following results :**

```
array([[ 1.,  0.,  0.,  0.],
       [ 0.,  1.,  0.,  0.],
       [ 0.,  0.,  1.,  0.],
       [ 0.,  0.,  0.,  1.]])
```

- `eye` **function returns 2-D array** :
  - Ones on the diagonal and zeros elsewhere, offset of diagonal can be set

```
np.eye(3, k=1)
```

```
array([[ 0.,  1.,  0.],
       [ 0.,  0.,  1.],
       [ 0.,  0.,  0.]])
```



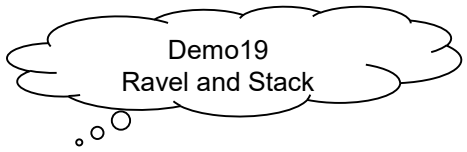
Ravel and Stack

```
x= np.array([(1,2,3),(3,4,5)])
print(x.ravel())    # [1 2 3 3 4 5]

x= np.array([(1,2,3),(3,4,5)])
y= np.array([(1,2,3),(3,4,5)])

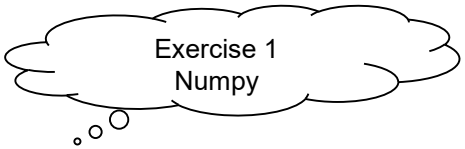
print(np.vstack((x,y)))
# [[1 2 3]
#  [3 4 5]
#  [1 2 3]
#  [3 4 5]]

print(np.hstack((x,y)))
# [[1 2 3 1 2 3]
#  [3 4 5 3 4 5]]
```



Summary : Numpy

- **Numpy is Python module for scientific computing :**
  - Python extension written in C used for fast vector and matrix operations
- **Core of NumPy package is `ndarray` object :**
  - Encapsulates n-dimensional arrays of homogeneous data types
- **NumPy supports vectorization and broadcasting :**
  - Vectorization is absence of explicit looping, indexing in code
  - Broadcasting refers to element-by-element behavior of operation
- **NumPy has much speed for element-by-element operations :**
  - Executed by pre-compiled C code
- **NumPy's `nparray` object is homogeneous multidimensional array :**
  - Dimensions are called axes, number of axes is rank
- **SciPy library depends on NumPy :**
  - Has variety of high level science and engineering modules
  - Statistics, linear algebra and signal processing
- **Pandas library also depends on NumPy**



© copyright : spiraltrain@gmail.com

SpiralTrain
developer courses

Course Schedule

- Numpy
- **Pandas Intro**
- Extracting and Transforming Data
- Advanced Indexing
- Rearranging Data
- Grouping Data
- MatplotLib
- Time Series

- **What are Pandas?**
- **Key Features of Pandas**
- **Pandas DataFrames**
- **Indexes and Columns**
- **Slicing**
- **head() and tail() Function**
- **info() and describe() Functions**
- **Broadcasting**
- **Series**
- **DataFrames from CVS Files**
- **DataFrame from Dictionary**
- **header Keyword**
- **names Keyword**
- **na_values Keyword**
- **parse_dates Keyword**
- **Inspecting DataFrames**
- **Using Dates as Index**
- **Trimming Redundant Columns**
- **Writing Files**

© copyright : spiraltrain@gmail.com

What are Pandas?

- `pandas` **is a Python library for data analysis :**
  - Provides fast, flexible and expressive data structures
- **High-level building block :**
  - Allows for doing practical, real world data analysis in Python
- `pandas` **are built on** `numpy` **which library must be loaded :**

```
import pandas as pd
import numpy as np
```

- **Pandas is well suited for :**
  - Tabular data with heterogeneously-typed columns as in SQL table or Excel spreadsheet
  - Ordered and unordered not necessarily fixed-frequency time series data
  - Arbitrary matrix data homogeneously typed or heterogeneous with row and column labels
  - Any other form of observational / statistical data sets
- **Data can be loaded from :**
  - From flat files, Excel files, databases and HDF5 (Hierarchical Data Format)
- **Pandas can provide meaningful labels**

Key Features of Pandas

- **High-performance containers for data analysis :**
  - Data structures with a lot of functionality
- **Intuitive merging and joining data sets :**
  - Easy handling of missing data
- **Size mutability :**
  - Columns can be inserted and deleted from DataFrame and higher dimensional objects
- **Automatic and explicit data alignment :**
  - Objects can be explicitly aligned to set of labels or data can be aligned automatically
- **Powerful, flexible group by functionality :**
  - To perform split-apply-combine operations on data sets
- **On large data sets :**
  - Intelligent label-based slicing, fancy indexing and subsetting, relational operations
- **Flexible reshaping and pivoting of data sets**
- **Time series functionality :**
  - Date range generation and frequency conversion, Date shifting and lagging,
  - Moving window statistics, moving window linear regressions, etc

Pandas Data Structures

- **Key building blocks :**
  - Indexes: Sequence of labels
  - Series: 1D array with Index
  - DataFrames: 2D array with Series as columns
- **Indexes**
  - Immutable like dictionary keys
  - Homogenous in data type like NumPy arrays

pandas DataFrames

- **Example: DataFrame of Apple Stock data**

aapl - DataFrame

Index	Date	Open	High	Low	Close	Adj Close	Volume
0	2014-09-29	100.59	100.69	98.04	99.62	93.5143	142718700
1	2014-10-06	99.95	102.38	98.31	100.73	94.5562	280258200
2	2014-10-13	101.33	101.78	95.18	97.67	91.6838	358539800
3	2014-10-20	98.32	105.49	98.22	105.22	98.771	358532900
4	2014-10-27	104.85	108.04	104.7	108	101.381	220230600
5	2014-11-03	108.22	110.3	107.72	109.01	102.329	199952900
6	2014-11-10	109.02	114.19	108.4	114.18	107.647	205166700
7	2014-11-17	114.27	117.57	113.3	116.47	109.806	233414700
8	2014-11-24	116.85	119.75	116.62	118.93	112.125	181873900
9	2014-12-01	118.81	119.25	111.27	115	108.42	266589700
10	2014-12-08	114.1	114.85	109.35	109.73	103.451	259868000

Indexes and Columns

```
import pandas as pd
file = 'AAPL.csv'
aapl = pd.read_csv(file)
type(aapl)

Out[1]: pandas.core.frame.DataFrame

aapl.shape
Out[2]: (184, 7)

aapl.columns
Out[3]: Index(['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close',
'Volume'], dtype='object')

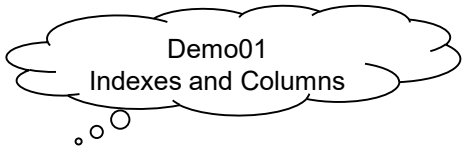
aapl.set_index('Date')
Out[4]:
                  Open        High    ...       Adj Close     Volume
Date                                  ...
2014-09-29  100.589996  100.690002    ...       93.514290  142718700
2014-10-06   99.949997  102.379997    ...       94.556244  280258200
```

Indexes and Columns

```
aapl.index
Out[5]: RangeIndex(start=0, stop=184, step=1)

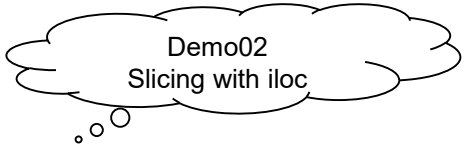
aapl = aapl.set_index('Date')

aapl.index
Out[7]:
Index(['2014-09-29', '2014-10-06', '2014-10-13', '2014-10-20', '2014-10-27',
       '2014-11-03', '2014-11-10', '2014-11-17', '2014-11-24', '2014-12-01',
       ...
       '2018-01-29', '2018-02-05', '2018-02-12', '2018-02-19', '2018-02-26',
       '2018-03-05', '2018-03-12', '2018-03-19', '2018-03-26', '2018-03-29'],
      dtype='object', name='Date', length=184)
type(aapl.index)
Out[8]: pandas.core.indexes.base.Index
```



Slicing with iloc

```
aapl.iloc[:5,:]
Out[9]:
                 Open        High     ...       Adj Close     Volume
Date                                  ...
2014-09-29  100.589996  100.690002     ...       93.514290  142718700
2014-10-06   99.949997  102.379997     ...       94.556244  280258200
2014-10-13  101.330002  101.779999     ...       91.683792  358539800
2014-10-20   98.320000  105.489998     ...       98.771042  358532900
2014-10-27  104.849998  108.040001     ...      101.380676  220230600
[5 rows x 6 columns]
aapl.iloc[-5:,:]
Out[10]:
                 Open        High     ...       Adj Close     Volume
Date                                  ...
2018-03-05  175.210007  180.000000     ...      179.979996  139852700
2018-03-12  180.289993  183.500000     ...      178.020004  155417500
2018-03-19  177.320007  177.470001     ...      164.940002  172670700
2018-03-26  168.070007  175.149994     ...      167.779999  157898400
2018-03-29  167.804993  171.750000     ...      167.779999   38398505
[5 rows x 6 columns]
```



head()

```
aapl.head(6)
Out[11]:
                 Open        High     ...       Adj Close     Volume
Date                                  ...
2014-09-29  100.589996  100.690002     ...       93.514290  142718700
2014-10-06   99.949997  102.379997     ...       94.556244  280258200
2014-10-13  101.330002  101.779999     ...       91.683792  358539800
2014-10-20   98.320000  105.489998     ...       98.771042  358532900
2014-10-27  104.849998  108.040001     ...      101.380676  220230600
2014-11-03  108.220001  110.300003     ...      102.328766  199952900

[6 rows x 6 columns]

aapl.head(2)
Out[12]:
                 Open        High     ...       Adj Close     Volume
Date                                  ...
2014-09-29  100.589996  100.690002     ...       93.514290  142718700
2014-10-06   99.949997  102.379997     ...       94.556244  280258200

[2 rows x 6 columns]
```

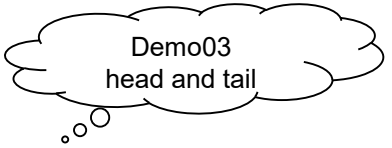
tail()

```
aapl.tail()
Out[13]:
                 Open        High     ...       Adj Close     Volume
Date                                  ...
2018-03-05  175.210007  180.000000     ...      179.979996  139852700
2018-03-12  180.289993  183.500000     ...      178.020004  155417500
2018-03-19  177.320007  177.470001     ...      164.940002  172670700
2018-03-26  168.070007  175.149994     ...      167.779999  157898400
2018-03-29  167.804993  171.750000     ...      167.779999   38398505

[5 rows x 6 columns]

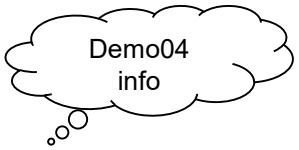
aapl.tail(3)
Out[14]:
                 Open        High     ...       Adj Close     Volume
Date                                  ...
2018-03-19  177.320007  177.470001     ...      164.940002  172670700
2018-03-26  168.070007  175.149994     ...      167.779999  157898400
2018-03-29  167.804993  171.750000     ...      167.779999   38398505

[3 rows x 6 columns]
```



info()

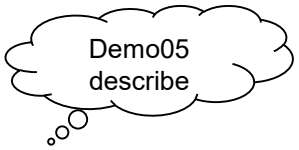
```
aapl.info()
<class 'pandas.core.frame.DataFrame'>
Index: 184 entries, 2014-09-29 to 2018-03-29
Data columns (total 6 columns):
Open          184 non-null float64
High          184 non-null float64
Low           184 non-null float64
Close         184 non-null float64
Adj Close     184 non-null float64
Volume        184 non-null int64
dtypes: float64(5), int64(1)
memory usage: 10.1+ KB
```



describe()

```
aapl.describe()
Out[16]:
              Open        High       ...         Adj Close        Volume
count   184.000000  184.000000       ...        184.000000  1.840000e+02
mean    127.040245  129.918098       ...        123.838032  1.910168e+08
std      24.314449   24.577906       ...         25.655569  8.160281e+07
min      92.389999   93.769997       ...         87.796822  3.839850e+07
25%     108.882500  112.362499       ...        105.301766  1.273706e+08
50%     118.895000  120.865002       ...        114.999730  1.744112e+08
75%     144.935005  147.695000       ...        143.039467  2.207755e+08
max     180.289993  183.500000       ...        179.979996  5.003630e+08

[8 rows x 6 columns]
```



Broadcasting

```
import numpy as np
```

- **Assigning scalar value to column slice broadcasts value to each row**

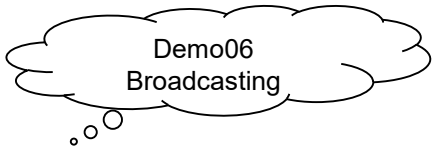
```
aapl.iloc[::3,-1] = np.nan
aapl.head(6)
Out[19]:
                 Open        High     ...         Adj Close       Volume
Date                                  ...
2014-09-29  100.589996  100.690002     ...         93.514290          NaN
2014-10-06   99.949997  102.379997     ...         94.556244  280258200.0
2014-10-13  101.330002  101.779999     ...         91.683792  358539800.0
2014-10-20   98.320000  105.489998     ...         98.771042          NaN
2014-10-27  104.849998  108.040001     ...        101.380676  220230600.0
2014-11-03  108.220001  110.300003     ...        102.328766  199952900.0

[6 rows x 6 columns]
```

info() After Broadcasting

- **Less Volume entries are non-null**

```
aapl.info()
<class 'pandas.core.frame.DataFrame'>
Index: 184 entries, 2014-09-29 to 2018-03-29
Data columns (total 6 columns):
Open          184 non-null float64
High          184 non-null float64
Low           184 non-null float64
Close         184 non-null float64
Adj Close     184 non-null float64
Volume        122 non-null float64
dtypes: float64(6)
memory usage: 10.1+ KB
```



Series

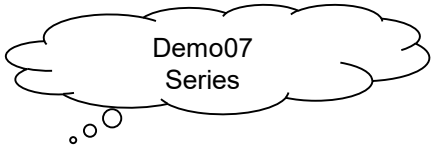
```
low = aapl['Low']

type(low)
Out[23]: pandas.core.series.Series

low.head()
Out[24]:
Date
2014-09-29     98.040001
2014-10-06     98.309998
2014-10-13     95.180000
2014-10-20     98.220001
2014-10-27    104.699997
Name: Low, dtype: float64

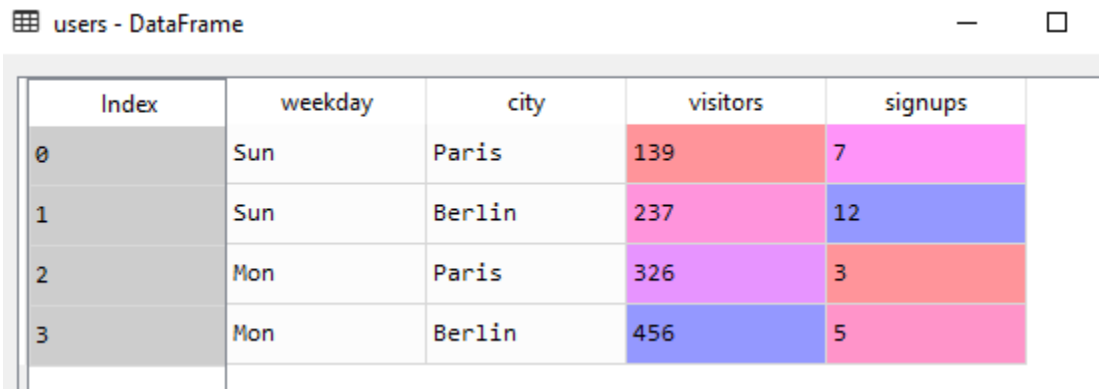
lows = low.values

type(lows)
Out[26]: numpy.ndarray
```



DataFrames from CSV files

```
import pandas as pd
users = pd.read_csv('users.csv', index_col=0)
users
Out[3]:
  weekday    city  visitors  signups
0     Sun   Paris       139        7
1     Sun  Berlin       237       12
2     Mon   Paris       326        3
3     Mon  Berlin       456        5
```

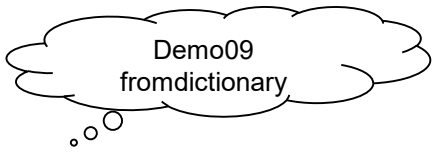


Index	weekday	city	visitors	signups
0	Sun	Paris	139	7
1	Sun	Berlin	237	12
2	Mon	Paris	326	3
3	Mon	Berlin	456	5

Demo08
fromcsv

DataFrames from Dictionary

```
import pandas as pd
data = {'weekday': ['Sun', 'Sun', 'Mon', 'Mon'],
        'city': ['Paris', 'Berlin', 'Paris', 'Berlin'],
        'visitors': [139, 237, 326, 456],
        'signups': [7, 12, 3, 5]}
data
Out[3]:
{'weekday': ['Sun', 'Sun', 'Mon', 'Mon'],
 'city': ['Paris', 'Berlin', 'Paris', 'Berlin'],
 'visitors': [139, 237, 326, 456],
 'signups': [7, 12, 3, 5]}
users = pd.DataFrame(data)
users
Out[5]:
  weekday    city  visitors  signups
0     Sun   Paris       139        7
1     Sun  Berlin       237       12
2     Mon   Paris       326        3
3     Mon  Berlin       456        5
```

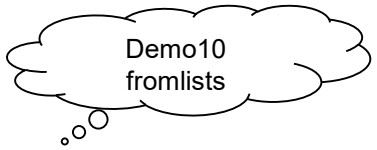


DataFrames from Lists

```
cities = ['Paris', 'Berlin', 'Paris', 'Berlin']
signups = [7, 12, 3, 5]
visitors = [139, 237, 326, 456]
weekdays = ['Sun', 'Sun', 'Mon', 'Mon']

list_labels = ['city', 'signups', 'visitors', 'weekday']
list_cols = [cities, signups, visitors, weekdays]
zipped = list(zip(list_labels, list_cols))

zipped
Out[13]:
[('city', ['Paris', 'Berlin', 'Paris', 'Berlin']),
 ('signups', [7, 12, 3, 5]),
 ('visitors', [139, 237, 326, 456]),
 ('weekday', ['Sun', 'Sun', 'Mon', 'Mon'])]
data = dict(zipped)
users = pd.DataFrame(data)
users
Out[16]:
     city  signups  visitors weekday
0   Paris        7       139     Sun
1  Berlin       12       237     Sun
2   Paris        3       326     Mon
```



Broadcasting New Column

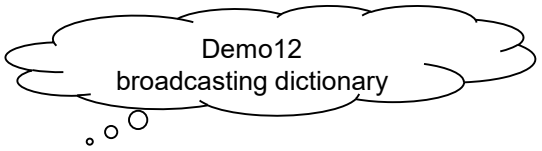
```
users['fees'] = 0

users
Out[18]:
     city  signups  visitors weekday  fees
0   Paris        7       139     Sun     0
1  Berlin       12       237     Sun     0
2   Paris        3       326     Mon     0
3  Berlin        5       456     Mon     0
```



Broadcasting with Dictionary

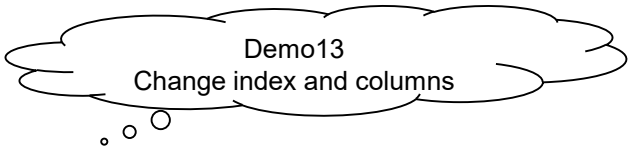
```
import pandas as pd
heights = [ 59.0, 65.2, 62.9, 65.4, 63.7, 65.7, 64.1 ]
data = {'height': heights, 'sex': 'M'}
results = pd.DataFrame(data)
results
Out[5]:
   height sex
0    59.0   M
1    65.2   M
2    62.9   M
3    65.4   M
4    63.7   M
5    65.7   M
6    64.1   M
```



Change Index and Columns

```
In [6]: results.columns = ['height (in)', 'sex']
In [7]: results.index = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
In [8]: print(results)

Out[8]:
  height (in) sex
A        59.0   M
B        65.2   M
C        62.9   M
D        65.4   M
E        63.7   M
F        65.7   M
G        64.1   M
```



Sunspot Observations CSV file

```
import pandas as pd
filepath = 'ISSN_D_tot.csv'
sunspots = pd.read_csv(filepath)
sunspots.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71860 entries, 0 to 71859
Data columns (total 6 columns):
1818        71860 non-null int64
01          71860 non-null int64
01.1        71860 non-null int64
1818.004    71860 non-null float64
 -1         71860 non-null int64
1           71860 non-null int64
dtypes: float64(1), int64(5)
memory usage: 3.3 MB
sunspots.iloc[10:13, :]
Out[5]:
    1818  01  01.1  1818.004   -1  1
10  1818   1    12  1818.034   -1  1
11  1818   1    13  1818.037   22  1
```



Dataset Problems

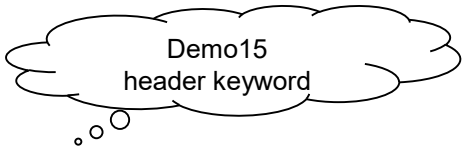
- **CSV file has no column headers :**
  - Columns 0-2: Gregorian date (year, month, day)
  - Column 3: Date as fraction as year
  - Column 4: Daily total sunspot number
  - Column 5: Definitive/provisional indicator (1 or 0)
- **Missing values in column 4: indicated by -1**
- **Dates representation is inconvenient**

header Keyword

```
sunspots = pd.read_csv(filepath, header=None)

sunspots.iloc[10:20, :]

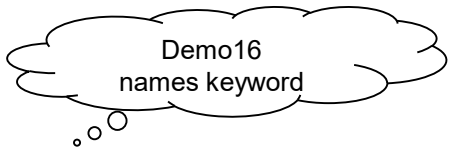
Out[7]:
       0  1   2          3   4  5
10  1818  1  11  1818.031  -1  1
11  1818  1  12  1818.034  -1  1
12  1818  1  13  1818.037  22  1
13  1818  1  14  1818.040  -1  1
14  1818  1  15  1818.042  -1  1
15  1818  1  16  1818.045  -1  1
16  1818  1  17  1818.048  46  1
17  1818  1  18  1818.051  59  1
18  1818  1  19  1818.053  63  1
19  1818  1  20  1818.056  -1  1
```



names Keyword

```
col_names = ['year', 'month', 'day', 'dec_date', 'sunspots', 'definite']
sunspots = pd.read_csv(filepath, header=None, names = col_names)

sunspots.iloc[10:20, :]
Out[10]:
    year  month  day  dec_date  sunspots  definite
10  1818      1   11  1818.031        -1         1
11  1818      1   12  1818.034        -1         1
12  1818      1   13  1818.037        22         1
13  1818      1   14  1818.040        -1         1
14  1818      1   15  1818.042        -1         1
15  1818      1   16  1818.045        -1         1
16  1818      1   17  1818.048        46         1
17  1818      1   18  1818.051        59         1
18  1818      1   19  1818.053        63         1
19  1818      1   20  1818.056        -1         1
```



na_values keyword

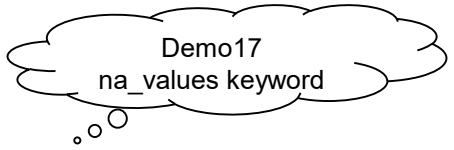
- **Note the space before -1**

```
sunspots = pd.read_csv(filepath, header=None,
                       names = col_names, na_values=' -1')
sunspots.iloc[10:20, :]
```

	year	month	day	dec_date	sunspots	definite
10	1818	1	11	1818.031	NaN	1
11	1818	1	12	1818.034	NaN	1
12	1818	1	13	1818.037	22.0	1
13	1818	1	14	1818.040	NaN	1
14	1818	1	15	1818.042	NaN	1
15	1818	1	16	1818.045	NaN	1
16	1818	1	17	1818.048	46.0	1
17	1818	1	18	1818.051	59.0	1
18	1818	1	19	1818.053	63.0	1
19	1818	1	20	1818.056	NaN	1

- **Can also use :**

```
sunspots = pd.read_csv(filepath, header=None,
     ...: names=col_names, na_values={'sunspots':[' -1']})
```



parse_dates Keyword

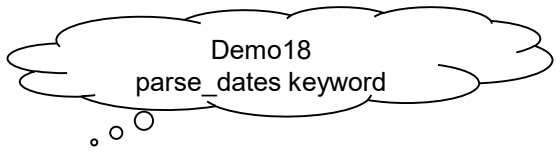
```
sunspots = pd.read_csv(filepath, header=None, names = col_names,
                       na_values={'sunspots':[' -1']}, parse_dates=[[0, 1, 2]])

sunspots.iloc[10:20, :]
Out[20]:
   year_month_day  dec_date  sunspots  definite
10     1818-01-11  1818.031       NaN         1
11     1818-01-12  1818.034       NaN         1
12     1818-01-13  1818.037      22.0         1
13     1818-01-14  1818.040       NaN         1
14     1818-01-15  1818.042       NaN         1
15     1818-01-16  1818.045       NaN         1
16     1818-01-17  1818.048      46.0         1
17     1818-01-18  1818.051      59.0         1
18     1818-01-19  1818.053      63.0         1
19     1818-01-20  1818.056       NaN         1
```

DataFrame Inspection

```
sunspots.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71861 entries, 0 to 71860
Data columns (total 4 columns):
year_month_day    71861 non-null datetime64[ns]
dec_date          71861 non-null float64
sunspots          68614 non-null float64
definite          71861 non-null int64
dtypes: datetime64[ns](1), float64(2), int64(1)
memory usage: 2.2 MB
```

- **Notice type of** `year_month_day` **column is** `datetime64`



Using Dates as Index

```
sunspots.index = sunspots['year_month_day']
sunspots.index.name = 'date'
sunspots.info()
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 71861 entries, 1818-01-01 to 2014-09-30
Data columns (total 4 columns):
year_month_day    71861 non-null datetime64[ns]
dec_date          71861 non-null float64
sunspots          68614 non-null float64
definite          71861 non-null int64
dtypes: datetime64[ns](1), float64(2), int64(1)
memory usage: 2.7 MB
```

```
sunspots.iloc[10:20, :]
Out[25]:
           year_month_day  dec_date  sunspots  definite
date
1818-01-11     1818-01-11  1818.031       NaN         1
1818-01-12     1818-01-12  1818.034       NaN         1
1818-01-13     1818-01-13  1818.037      22.0         1
```

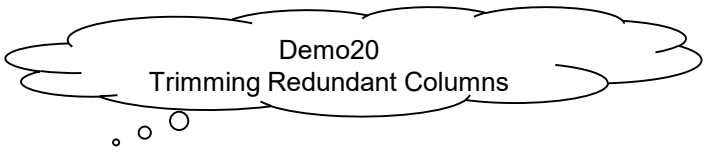


Trimming Redundant Columns

```
cols = ['sunspots', 'definite']

sunspots = sunspots[cols]

sunspots.iloc[10:20, :]
Out[28]:
            sunspots  definite
date
1818-01-11       NaN         1
1818-01-12       NaN         1
1818-01-13      22.0         1
1818-01-14       NaN         1
1818-01-15       NaN         1
1818-01-16       NaN         1
1818-01-17      46.0         1
1818-01-18      59.0         1
1818-01-19      63.0         1
1818-01-20       NaN         1
```

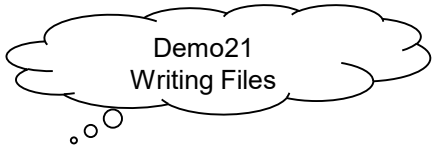


Writing files

```
out_csv = 'sunspots.csv'
sunspots.to_csv(out_csv)

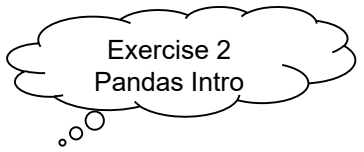
out_tsv = 'sunspots.tsv'
sunspots.to_csv(out_tsv, sep='\t')

out_xlsx = 'sunspots.xlsx'
sunspots.to_excel(out_xlsx)
```



Summary: Pandas Intro

- `pandas` **is a Python library for data analysis :**
  - Provides fast, flexible and expressive data structures
- **High-level building block :**
  - `pandas` are built on `numpy` which library must be present
- **Key building blocks of** `pandas` **are :**
  - Indexes, Series: 1D array with Index, DataFrames: 2D array with Series as columns
- **Indexes are sequence of labels**
  - Immutable like dictionary keys, homogenous in data type like `numpy` arrays
- **Key Features of Pandas :**
  - Flexible reshaping and pivoting of data sets
  - Powerful, flexible group by functionality
  - Flexible reshaping and pivoting of data sets
- **Data can be loaded in Pandas from :**
  - From flat files like csv files, Excel files, databases and HDF5



Course Schedule

- Numpy
- Pandas Intro
- **Extracting and Transforming Data**
- Advanced Indexing
- Rearranging Data
- Grouping Data
- MatplotLib
- Time Series

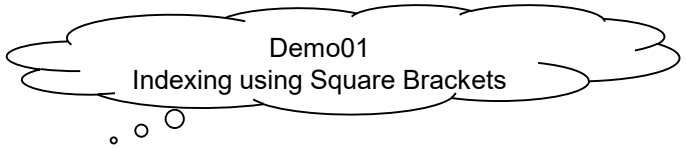
- **Indexing Data Frames**
- **Using Column Attribute**
- **Using Row Label**
- **.loc and .iloc Accessor**
- **Selecting Columns**
- **Slicing and Indexing a Series**
- **Using Lists**
- **DataFrame with 1 Column**
- **Boolean Series**
- **Filtering with Boolean Series**
- **Combining Filters**
- **Zeros and NaNs**
- **all and any Nonzeros**
- **any NaNs and without NaNs**
- **Drop Rows with any NaNs**
- **Modifying Columns**
- **Vectorized Methods**
- **Storing Transformations**

© copyright : spiraltrain@gmail.com

Indexing using Square Brackets

```
In [1]: import pandas as pd
In [2]: df = pd.read_csv('sales.csv', index_col='month')
In [3]: df
Out[3]:
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
May       132     NaN        52
Jun       205     60.0       55

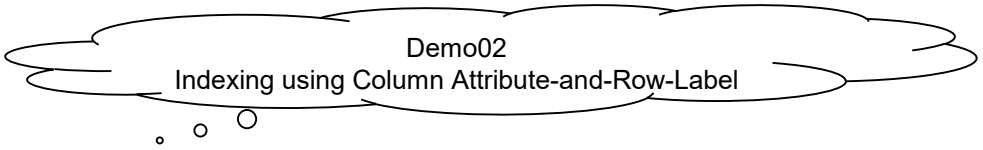
In [4]: df['salt']['Jan']
Out[4]: 12.0
```



Using Column Attribute and Row Label

```
In [6]: df
Out[6]:
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
May       132     NaN        52
Jun       205     60.0       55

In [7]: df.eggs['Mar']
Out[7]: 221
```

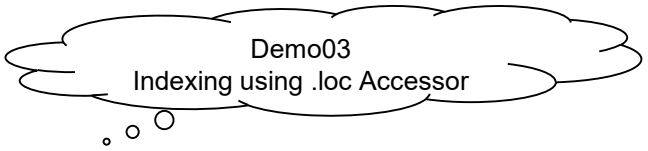


Using the .loc Accessor

```
In [8]: df
Out[8]:

         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
May       132     NaN        52
Jun       205     60.0       55


In [9]: df.loc['May', 'spam']
Out[9]: 52.0
```

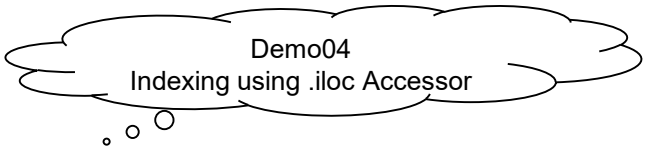


Using the .iloc Accessor

```
In [10]: df
Out[10]:

         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
May       132     NaN        52
Jun       205     60.0       55

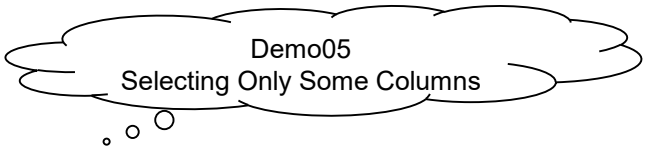
In [11]: df.iloc[4, 2]
Out[11]: 52.0
```



Selecting only Some Columns

```
In [12]: df_new = df[['salt','eggs']]
In [13]: df_new
Out[13]:

         salt     eggs
month
Jan      12.0       47
Feb      50.0      110
Mar      89.0      221
Apr      87.0       77
May       NaN      132
Jun      60.0      205
```

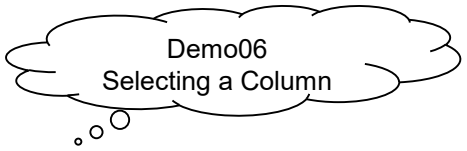


Selecting a Column

```
In [2]: df['eggs']
Out[2]:

month
Jan       47
Feb      110
Mar      221
Apr       77
May      132
Jun      205
Name: eggs, dtype: int64

In [3]: type(df['eggs'])
Out[3]: pandas.core.series.Series
```

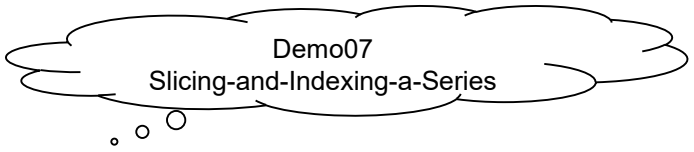


Slicing and Indexing a Series

```
In [4]: df['eggs'][1:4] # Part of the eggs column
Out[4]:

month
Feb      110
Mar      221
Apr       77
Name: eggs, dtype: int64

In [5]: df['eggs'][4] # The value associated with May
Out[5]: 132
```



Using .loc[]

```
In [6]: df.loc[:, 'eggs':'salt'] # All rows, some columns
Out[6]:
         eggs     salt
month
Jan        47     12.0
Feb       110     50.0
Mar       221     89.0
Apr        77     87.0
May       132      NaN
Jun       205     60.0
```

```
In [7]: df.loc['Jan':'Apr',:] # Some rows, all columns
Out[7]:
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55

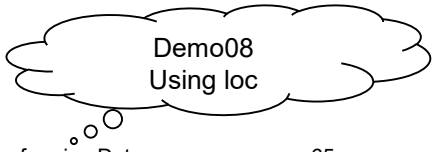
Using .loc[]

```
In [8]: df.loc['Mar':'May', 'salt':'spam']
Out[8]:
         salt     spam
month
Mar      89.0       72
Apr      87.0       20
May      NaN        52
```

```
In [9]: df.iloc[2:5, 1:] # A block from middle of the DataFrame
Out[9]:
         salt     spam
month
Mar      89.0       72
Apr      87.0       20
May      NaN        52
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55



Using Lists rather than Slices

```
In [10]: df.loc['Jan':'May', ['eggs', 'spam']]
Out[10]:
         eggs     spam
month
Jan        47       17
Feb       110       31
Mar       221       72
Apr        77       20
May       132       52

In [11]: df.iloc[[0,4,5], 0:2]
Out[11]:
         eggs     salt
month
Jan        47     12.0
May       132      NaN
Jun       205     60.0
```

Original

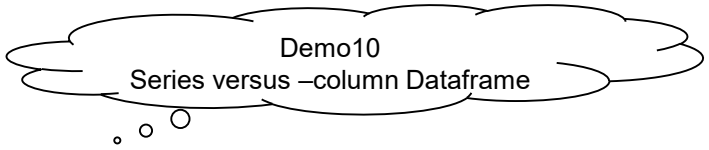
month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55

Demo09
Using Lists rather than Slices

Series versus 1-column DataFrame

```
# A Series by column name
In [13]: df['eggs']
Out[13]:
month
Jan       47
Feb      110
Mar      221
Apr       77
May      132
Jun      205
Name: eggs, dtype: int64
In [14]: type(df['eggs'])
Out[14]:
pandas.core.series.Series
```

```
# A DataFrame with single column
In [15]: df[['eggs']]
Out[15]:
         eggs
month
Jan        47
Feb       110
Mar       221
Apr        77
May       132
Jun       205
In [16]: type(df[['eggs']])
Out[16]:
pandas.core.frame.DataFrame
```



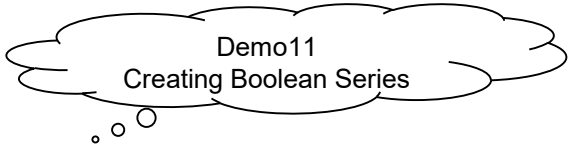
Creating a Boolean Series

```
In [1]: df.salt > 60
Out[1]:

month
Jan      False
Feb      False
Mar       True
Apr       True
May      False
Jun      False
Name: salt, dtype: bool
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55



Filtering with a Boolean Series

```
In [2]: df[df.salt > 60]
Out[2]:

         eggs     salt     spam
month
Mar       221     89.0       72
Apr        77     87.0       20

In [3]: enough_salt_sold = df.salt > 60

In [4]: df[enough_salt_sold]
Out[4]:
         eggs     salt     spam
month
Mar       221     89.0       72
Apr        77     87.0       20
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55

Demo12
Filtering with Boolean Series

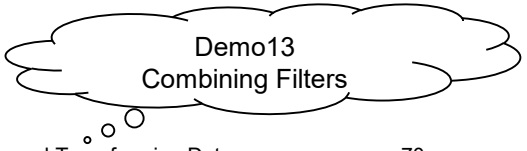
Combining Filters

```
In [5]: df[(df.salt >= 50) & (df.eggs < 200)] # Both conditions
Out[5]:
         eggs     salt     spam
month
Feb       110     50.0       31
Apr        77     87.0       20

In [6]: df[(df.salt >= 50) | (df.eggs < 200)] # Either condition
Out[6]:
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
May       132      NaN       52
Jun       205     60.0       55
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55



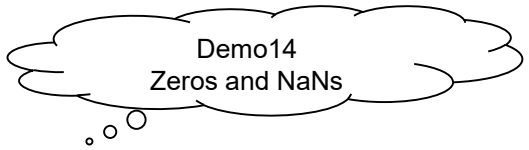
Zeros and NaNs

```
In [7]: df2 = df.copy()
In [8]: df2['bacon'] = [0, 0, 50, 60, 70, 80]
In [9]: df2
Out[9]:
```

month	eggs	salt	spam	bacon
Jan	47	12.0	17	0
Feb	110	50.0	31	0
Mar	221	89.0	72	50
Apr	77	87.0	20	60
May	132	NaN	52	70
Jun	205	60.0	55	80

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55



all and any Nonzeros

```
In [10]: df2.loc[:, df2.all()]
Out[10]:
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
May       132     NaN        52
Jun       205     60.0       55
In [11]: df2.loc[:, df2.any()]
Out[11]:
         eggs     salt     spam     bacon
month
Jan        47     12.0       17         0
Feb       110     50.0       31         0
Mar       221     89.0       72        50
Apr        77     87.0       20        60
May       132      NaN       52        70
Jun       205     60.0       55        80
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55

Demo15
All and Any non zeros

any NaNs and without NaNs

```
In [12]: df.loc[:, df.isnull().any()]
Out[12]:
         salt
month
Jan      12.0
Feb      50.0
Mar      89.0
Apr      87.0
May       NaN
Jun      60.0
In [13]: df.loc[:, df.notnull().all()]
Out[13]:
         eggs     spam
month
Jan        47       17
Feb       110       31
Mar       221       72
Apr        77       20
May       132       52
Jun       205       55
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55

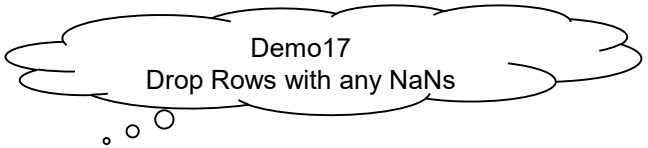
Demo16
Any NaN without NaN

Drop Rows with any NaNs

```
In [14]: df.dropna(how='any')
Out[14]:
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
Jun       205     60.0       55
```

Original

```
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
May       132     NaN        52
Jun       205     60.0       55
```



Filtering and Modifying Column based on Another

```
In [15]: df.eggs[df.salt > 55]
Out[15]:
month
Mar      221
Apr       77
Jun      205
Name: eggs, dtype: int64

In [16]: df.eggs[df.salt > 55] += 5
In [17]: df
Out[17]:
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       226     89.0       72
Apr        82     87.0       20
May       132     NaN        52
Jun       210     60.0       55
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55

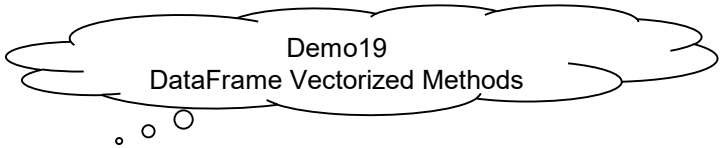
Demo18
Filtering and Modifying Column based on Another

DataFrame Vectorized Methods

```
In [1]: df.floordiv(12) # Convert to dozens unit
Out[1]:
         eggs     salt     spam
month
Jan         3     1.0         1
Feb         9     4.0         2
Mar        18     7.0         6
Apr         6     7.0         1
May        11     NaN         4
Jun        17     5.0         4
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55



NumPy Vectorized Functions

```
In [2]: import numpy as np
In [3]: np.floor_divide(df, 12) # Convert to dozens unit
Out[3]:
         eggs     salt     spam
month
Jan       3.0     1.0       1.0
Feb       9.0     4.0       2.0
Mar      18.0     7.0       6.0
Apr       6.0     7.0       1.0
May      11.0     NaN       4.0
Jun      17.0     5.0       4.0
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55

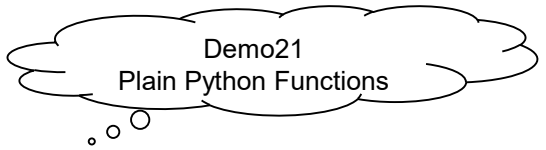
Demo20
Numpy Vectorized Functions

Plain Python Functions

```
In [4]: def dozens(n):
....: return n//12
In [5]: df.apply(dozens) # Convert to dozens unit
Out[5]:
         eggs     salt     spam
month
Jan         3     1.0         1
Feb         9     4.0         2
Mar        18     7.0         6
Apr         6     7.0         1
May        11     NaN         4
Jun        17     5.0         4
In [6]: df.apply(lambda n: n//12)
Out[6]:
         eggs     salt     spam
month
Jan         3     1.0         1
Feb         9     4.0         2
Mar        18     7.0         6
Apr         6     7.0         1
May        11     NaN         4
Jun        17     5.0         4
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55



Storing a Transformation

```
In [7]: df['dozens_of_eggs'] = df.eggs.floordiv(12)
In [8]: df
Out[8]:
         eggs     salt     spam     dozens_of_eggs
month
Jan        47     12.0       17      3
Feb       110     50.0       31      9
Mar       221     89.0       72     18
Apr        77     87.0       20      6
May       132     NaN        52     11
Jun       205     60.0       55     17
```

Original

```
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
May       132     NaN        52
Jun       205     60.0       55
```

Demo22
Storing a Transformation

Exercise 3
Extracting and transforming data

SpiralTrain
developer courses

Course Schedule

- Numpy
- Pandas Intro
- Extracting and Transforming Data
- **Advanced Indexing**
- Rearranging Data
- Grouping Data
- MatplotLib
- Time Series

- **DataFrame Index**
- **Working with String Values**
- **Using map Function**
- **Defining Columns on Other Columns**
- **Creating a Series**
- **Non Mutability**
- **Modifying Index Entries**
- **Hierarchical Indexing**
- **Setting the Index**
- **Sorting Index**
- **Indexing Individual Row**
- **Slicing Outermost Index**
- **Fancy Indexing**

© copyright : spiraltrain@gmail.com

The DataFrame Index

```
         eggs     salt     spam     dozens_of_eggs
month
Jan        47     12.0       17      3
Feb       110     50.0       31      9
Mar       221     89.0       72     18
Apr        77     87.0       20      6
May       132     NaN        52     11
Jun       205     60.0       55     17
In [10]: df.index
Out[10]: Index(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
dtype='object', name='month')
```

Original

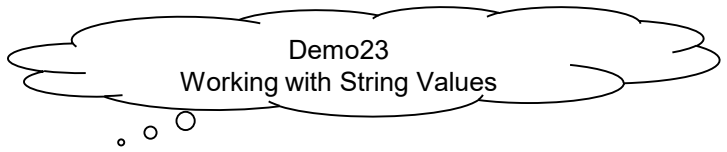
month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55

Working with String Values

```
In [11]: df.index = df.index.str.upper()
In [12]: df
Out[12]:
         eggs     salt     spam     dozens_of_eggs
month
JAN        47     12.0       17      3
FEB       110     50.0       31      9
MAR       221     89.0       72     18
APR        77     87.0       20      6
MAY       132      NaN       52     11
JUN       205     60.0       55     17
In [13]: df.index = df.index.map(str.lower)
In [14]: df
Out[14]:
         eggs     salt     spam     dozens_of_eggs
jan        47     12.0       17      3
feb       110     50.0       31      9
mar       221     89.0       72     18
apr        77     87.0       20      6
may       132     NaN        52     11
```

Original

month	eggs	salt	spam
Jan	47	12.0	17
Feb	110	50.0	31
Mar	221	89.0	72
Apr	77	87.0	20
May	132	NaN	52
Jun	205	60.0	55



Defining Columns using Other Columns

```
In [15]: df['salty_eggs'] = df.salt + df.dozens_of_eggs
In [16]: df
Out[16]:
         eggs     salt     spam     dozens_of_eggs    salty_eggs
jan        47     12.0       17      3                15.0
feb       110     50.0       31      9                59.0
mar       221     89.0       72     18               107.0
apr        77     87.0       20      6                93.0
may       132     NaN        52     11                NaN
jun       205     60.0       55     17                77.0
```

Original

```
         eggs     salt     spam
month
Jan        47     12.0       17
Feb       110     50.0       31
Mar       221     89.0       72
Apr        77     87.0       20
May       132     NaN        52
Jun       205     60.0       55
```

Demo24
Defining Columns using Other Columns

Creating a Series

```
In [1]: import pandas as pd
In [2]: prices = [10.70, 10.86, 10.74, 10.71, 10.79]
In [3]: shares = pd.Series(prices)
In [4]: print(shares)
0       10.70
1       10.86
2       10.74
3       10.71
4       10.79
dtype: float64
In [5]: days = ['Mon', 'Tue', 'Wed', 'Thur', 'Fri']
In [6]: shares = pd.Series(prices, index=days)
In [7]: print(shares)
Mon     10.70
Tue     10.86
Wed     10.74
Thur    10.71
Fri     10.79
dtype: float64
```

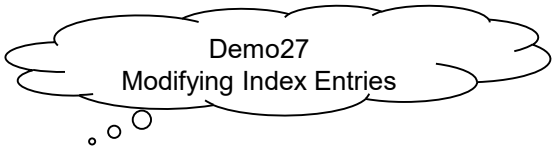


Modifying Index Entries

```
In [15]: shares.index[2] = 'Wednesday'
_____________________________________________

TypeError: Index does not support mutable operations
In [16]: shares.index[:4] = ['Monday', 'Tuesday',
...: 'Wednesday', 'Thursday']
_____________________________________________

TypeError: Index does not support mutable operations
In [17]: shares.index = ['Monday', 'Tuesday', 'Wednesday',
...: 'Thursday', 'Friday']
In [18]: print(shares)
Monday           10.70
Tuesday          10.86
Wednesday        10.74
Thursday         10.71
Friday           10.79
dtype: float64
```

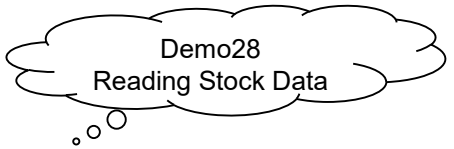


Hierarchical Indexing : Reading Stock Data

```
In [1]: import pandas as pd
In [2]: stocks = pd.read_csv('stocks.csv')
In [3]: print(stocks)
         Date     Close       Volume         Symbol
0  2016-10-03     31.50    14070500           CSCO
1  2016-10-03    112.52    21701800           AAPL
2  2016-10-03     57.42    19189500           MSFT
3  2016-10-04    113.00    29736800           AAPL
4  2016-10-04     57.24    20085900           MSFT
5  2016-10-04     31.35    18460400           CSCO
6  2016-10-05     57.64    16726400           MSFT
7  2016-10-05     31.59    11808600           CSCO
8  2016-10-05    113.05    21453100           AAPL
```

Repeated values

Repeated values



Set Index

```
In [4]: stocks = stocks.set_index(['Symbol', 'Date'])
In [5]: print(stocks)
                           Close    Volume
Symbol        Date
CSCO     2016-10-03        31.50  14070500
AAPL     2016-10-03       112.52  21701800
MSFT     2016-10-03        57.42  19189500
AAPL     2016-10-04       113.00  29736800
MSFT     2016-10-04        57.24  20085900
CSCO     2016-10-04        31.35  18460400
MSFT     2016-10-05        57.64  16726400
CSCO     2016-10-05        31.59  11808600
AAPL     2016-10-05       113.05  21453100

In [6]: print(stocks.index.name)
None
In [7]: print(stocks.index.names)
['Symbol', 'Date']
```

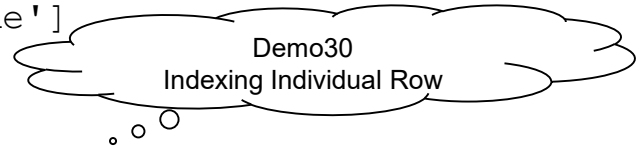
Sorting Index

```
In [9]: stocks = stocks.sort_index()
In [10]: print(stocks)
                      Close         Volume
Symbol   Date
AAPL     2016-10-03   112.52       21701800
         2016-10-04   113.00       29736800
         2016-10-05   113.05       21453100
CSCO     2016-10-03    31.50       14070500
         2016-10-04    31.35       18460400
         2016-10-05    31.59       11808600
MSFT     2016-10-03    57.42       19189500
         2016-10-04    57.24       20085900
         2016-10-05    57.64       16726400
```



Indexing Individual Row

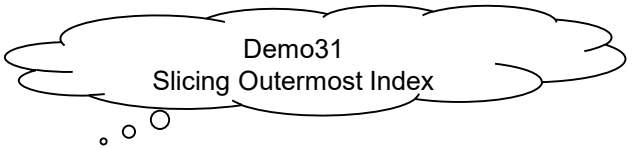
```
                        Close         Volume
Symbol   Date
AAPL     2016-10-03   112.52       21701800
         2016-10-04   113.00       29736800
         2016-10-05   113.05       21453100
CSCO     2016-10-03    31.50       14070500
         2016-10-04    31.35       18460400
         2016-10-05    31.59       11808600
MSFT     2016-10-03    57.42       19189500
         2016-10-04    57.24       20085900
         2016-10-05    57.64       16726400
In [11]: stocks.loc[('CSCO', '2016-10-04')]
Out[11]:
Close             31.35
Volume      18460400.00
Name: (CSCO, 2016-10-04), dtype: float64
In [12]: stocks.loc[('CSCO', '2016-10-04'), 'Volume']
Out[12]: 18460400.0
```



Slicing Outermost Index

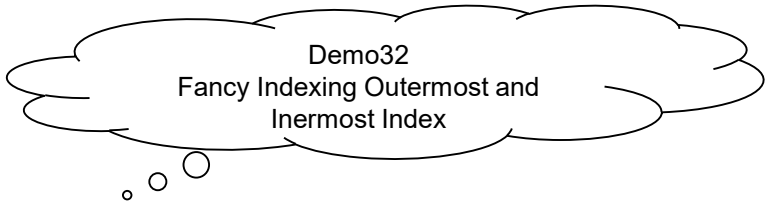
```
In [13]: stocks.loc['AAPL']
Out[13]:
                 Close    Volume
Date
2016-10-03       112.52  21701800
2016-10-04       113.00  29736800
2016-10-05       113.05  21453100

In [14]: stocks.loc['CSCO':'MSFT']
Out[14]:
                        Close     Volume
Symbol   Date
CSCO     2016-10-03      31.50   14070500
         2016-10-04      31.35   18460400
         2016-10-05      31.59   11808600
MSFT     2016-10-03      57.42   19189500
         2016-10-04      57.24   20085900
         2016-10-05      57.64   16726400
```

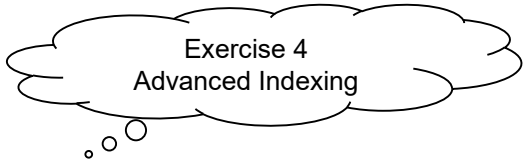


Fancy Indexing Outermost and Innermost Index

```
In [15]: stocks.loc[(['AAPL', 'MSFT'], '2016-10-05'), :]
Out[15]:
                          Close      Volume
Symbol   Date
AAPL     2016-10-05       113.05   21453100
MSFT     2016-10-05       57.64    16726400
In [16]: stocks.loc[(['AAPL', 'MSFT'], '2016-10-05'), 'Close']
Out[16]:
Symbol   Date
AAPL     2016-10-05       113.05
MSFT     2016-10-05       57.64
Name: Close, dtype: float64
```




```
In [17]: stocks.loc[('CSCO', ['2016-10-05', '2016-10-03']), :]
Out[17]:
                          Close     Volume
Symbol   Date
CSCO     2016-10-03       31.50   14070500
         2016-10-05       31.59   11808600
```



Course Schedule

- Numpy
- Pandas Intro
- Extracting and Transforming Data
- Advanced Indexing
- **Rearranging Data**
- Grouping Data
- MatplotLib
- Time Series

- **Reshaping by Pivoting**
- **pivot Function**
- **Pivoting Multiple Columns**
- **Creating Multi-level Index**
- **Unstacking Multi-Index**
- **Stacking DataFrames**
- **Swapping Levels**
- **Sorting Rows**
- **Rearranging by Pivoting**
- **Duplicate Entries**
- **Pivot Table**
- **Other Aggregations**
- **aggfunc Function**

© copyright : spiraltrain@gmail.com

Reshaping by Pivoting

- **Pivotting reshapes data and highlights desired information :**
  - Consists of row, column and data fields

```
In [1]: import pandas as pd
In [2]: trials = pd.read_csv('trials_01.csv')
In [3]: print(trials)
   id     treatment  gender response
0   1             A       F        5
1   2             A       M        3
2   3             B       F        8
3   4             B       M        9
```

- **Created with `pivot` function :**

```
In [4]: trials.pivot(index='treatment',
   ...: columns='gender',
   ...: values='response')
Out[4]:
gender     F M
treatment
A          5 3
B          8 9
```

Original

	id	treatment	gender	response
0	1	A	F	5
1	2	A	M	3
2	3	B	F	8
3	4	B	M	9

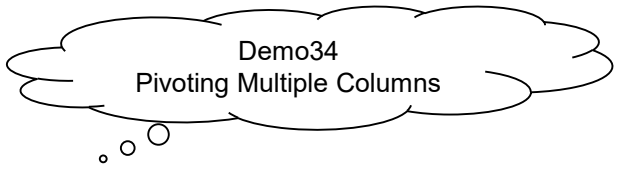
Demo33
Reshaping by Pivoting

Pivoting Multiple Columns

```
In [5]: trials.pivot(index='treatment', columns='gender')
Out[5]:
           id      response
gender     F M     F M
treatment
A          1 2     5 3
B          3 4     8 9
```

Original

	id	treatment	gender	response
0	1	A	F	5
1	2	A	M	3
2	3	B	F	8
3	4	B	M	9



Creating Multi-level Index

```
In [1]: print(trials)
  id treatment gender response
0  1         A      F        5
1  2         A      M        3
2  3         B      F        8
3  4         B      M        9

In [2]: trials = trials.set_index(['treatment', 'gender'])
In [3]: print(trials)
                  id response
treatment gender
A         F        1        5
          M        2        3
B         F        3        8
          M        4        9
```

Original

	id	treatment	gender	response
0	1	A	F	5
1	2	A	M	3
2	3	B	F	8
3	4	B	M	9

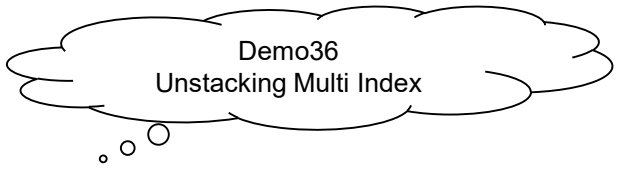
Demo35
Creating Multi Level Index

Unstacking a Multi-Index

```
In [4]: print(trials)
                  id response
treatment gender
A         F       1  5
          M       2  3
B         F       3  8
          M       4  9
In [5]: trials.unstack(level='gender')
Out[5]:
             id     response
gender       F M    F M
treatment
A            1 2    5 3
B            3 4    8 9
In [6]: trials.unstack(level=1)
Out[6]:
            id    response
gender      F M       F M
treatment
A           1 2       5 3
B           3 4       8 9
```

Original

	id	treatment	gender	response
0	1	A	F	5
1	2	A	M	3
2	3	B	F	8
3	4	B	M	9

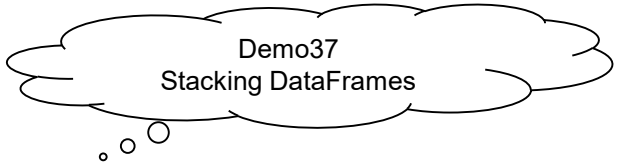


Stacking DataFrames

```
In [8]: trials_by_gender = trials.unstack(level='gender')
In [9]: trials_by_gender
Out[9]:
           id    response
gender      F M        F M
treatment
A           1 2        5 3
B           3 4        8 9
In [10]: trials_by_gender.stack(level='gender')
Out[10]:
                    id  response
treatment gender
A         F          1         5
          M          2         3
B         F          3         8
          M          4         9
```

Original

	id	treatment	gender	response
0	1	A	F	5
1	2	A	M	3
2	3	B	F	8
3	4	B	M	9



Swapping Levels and Sorting Rows

```
In [13]: swapped = stacked.swaplevel(0, 1)
In [14]: print(swapped)
                  id response
gender  treatment
F        A         1        5
M        A         2        3
F        B         3        8
M        B         4        9

In [15]: sorted_trials = swapped.sort_index()
In [16]: print(sorted_trials)
                  id response
gender  treatment
F        A         1        5
         B         3        8
M        A         2        3
         B         4        9
```

Original

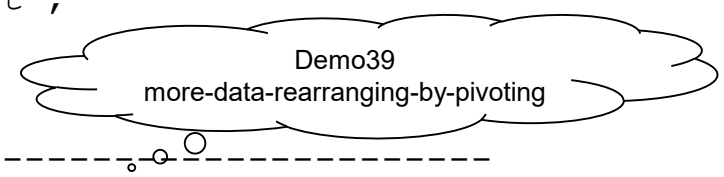
	id	treatment	gender	response
0	1	A	F	5
1	2	A	M	3
2	3	B	F	8
3	4	B	M	9

Demo38
Swapping Levels and Sorting Rows

More Data Rearranging by Pivoting

```
In [1]: import pandas as pd
In [2]: more_trials = pd.read_csv('trials_03.csv')
In [3]: print(more_trials)
   id  treatment gender response
0   1          A      F        5
1   2          A      M        3
2   3          A      M        8
3   4          A      F        9
4   5          B      F        1
5   6          B      M        8
6   7          B      F        4
7   8          B      F        6

In [4]: more_trials.pivot(index='treatment',
   ...: columns='gender',
   ...: values='response')
---------------------------------------------------------------------------
ValueError: Index contains duplicate entries, cannot reshape
```

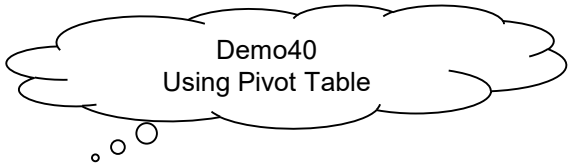


Pivot Table

- **Problem can be solved by using `pivot_table` instead of `pivot` :**
  - Now index stays unique and values are summarized

```
In [5]: more_trials.pivot_table(index='treatment',
   ...: columns='gender',
   ...: values='response')
Out[5]:
gender              F   M
treatment
A           7.000000 5.5
B           3.666667 8.0
```

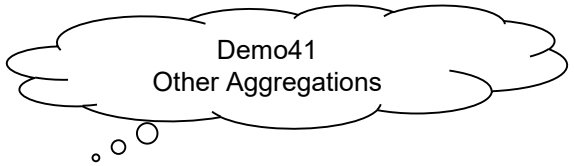
	id	treatment	gender	response
0	1	A	F	5
1	2	A	M	3
2	3	A	M	8
3	4	A	F	9
4	5	B	F	1
5	6	B	M	8
6	7	B	F	4
7	8	B	F	6

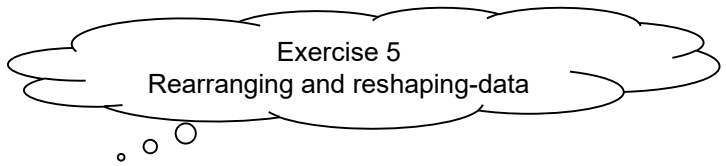


Other Aggregations

```
In [6]: more_trials.pivot_table(index='treatment',
   ...:                         columns='gender',
   ...:                         values='response',
   ...:                         aggfunc='count')
Out[6]:
gender             F  M
treatment
A                  2  2
B                  3  1
```

```
  id  treatment gender response
0   1          A      F        5
1   2          A      M        3
2   3          A      M        8
3   4          A      F        9
4   5          B      F        1
5   6          B      M        8
6   7          B      F        4
7   8          B      F        6
```





Course Schedule

- Numpy
- Pandas Intro
- Extracting and Transforming Data
- Advanced Indexing
- Rearranging Data
- **Grouping Data**
- MatplotLib
- Time Series

- **Boolean Filter and Count**
- **Aggregation and Reduction**
- **Groupby and sum**
- **Groupby and mean**
- **Groupby and sum**
- **Categorical Data**
- **groupBy Multiple Aggregations**
- **Custom Aggregation**
- **Groupby and Transformation**
- **Apply zscore**
- **Transformation and Aggregation**
- **Groupby and Filtering**
- **Groupby Object**
- **Iteration and Filtering**
- **Comprehension**
- **Boolean Groupby**

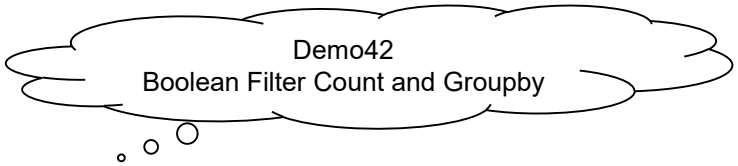
© copyright : spiraltrain@gmail.com

Categoricals and groupby : Boolean Filter and Count

```
In [1]: sales = pd.DataFrame(
   ...: {
   ...:  'weekday': ['Sun', 'Sun', 'Mon', 'Mon'],
   ...:  'city': ['Austin', 'Dallas', 'Austin', 'Dallas'],
   ...:  'bread': [139, 237, 326, 456],
   ...:  'butter': [20, 45, 70, 98]
   ...:  }
   ...: )

In [2]: sales.loc[sales['weekday'] == 'Sun'].count()
Out[2]:
bread      2
butter     2
city       2
weekday    2
dtype: int64
In [4]: sales.groupby('weekday').count()
Out[4]:
         bread butter city
weekday
Mon          2      2    2
Sun          2      2    2
```

`sales.groupby('weekday').count()` :
- split by 'weekday'
- apply count() function on each group
- combine counts per group



Aggregation and Reduction

- **Some reducing functions :**
  - `mean()`
  - `std()`
  - `sum()`
  - `first(), last()`
  - `min(), max()`

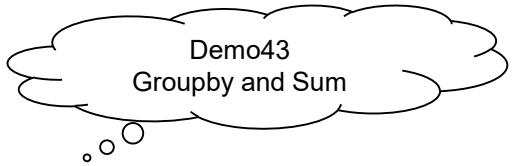
Groupby and sum, multiple columns

```
In [5]: sales.groupby('weekday')['bread'].sum()
Out[5]:
weekday
Mon    782
Sun    376
Name: bread, dtype: int64

In [6]: sales.groupby('weekday')[['bread','butter']].sum()
Out[6]:
         bread butter
weekday
Mon        782    168
Sun        376     65
```

original

	bread	butter	city	weekday
0	139	20	Austin	Sun
1	237	45	Dallas	Sun
2	326	70	Austin	Mon
3	456	98	Dallas	Mon

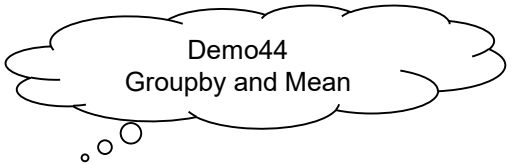


Groupby and Mean: Multi-Level Index

```
In [7]: sales.groupby(['city','weekday']).mean()
Out[7]:
                bread butter
city    weekday
Austin  Mon       326     70
        Sun       139     20
Dallas  Mon       456     98
        Sun       237     45
```

original

	bread	butter	city	weekday
0	139	20	Austin	Sun
1	237	45	Dallas	Sun
2	326	70	Austin	Mon
3	456	98	Dallas	Mon



Groupby and Sum: by Series

```
In [8]: customers = pd.Series(['Dave','Alice','Bob','Alice'])
In [9]: customers
Out[9]:
0    Dave
1   Alice
2     Bob
3   Alice

dtype: object
In [10]: sales.groupby(customers)['bread'].sum()
Out[10]:
Alice    693
Bob      326
Dave     139
Name: bread, dtype: int64
```

```
In [9]: customers
Out[9]:
0    Dave
1   Alice
2     Bob
3   Alice
dtype: object
```

original

	bread	butter	city	weekday
0	139	20	Austin	Sun
1	237	45	Dallas	Sun
2	326	70	Austin	Mon
3	456	98	Dallas	Mon

Demo45
Groupby and Sum by Series

Categorical Data

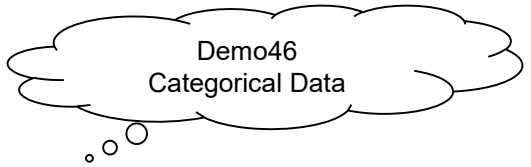
- **Category is pandas data type for categorical variables in statistics :**
  - Categorical variable takes on a limited number of possible values

```
In [11]: sales['weekday'].unique()
Out[11]: array(['Sun', 'Mon'], dtype=object)
In [12]: sales['weekday'] = sales['weekday'].astype('category')
In [13]: sales['weekday']
Out[13]:
0        Sun
1        Sun
2        Mon
3        Mon
Name: weekday, dtype: category
Categories (2, object): [Mon, Sun]
```

original

	bread	butter	city	weekday
0	139	20	Austin	Sun
1	237	45	Dallas	Sun
2	326	70	Austin	Mon
3	456	98	Dallas	Mon

- **Advantages :**
  - Uses less memory
  - Speeds up operations like `groupby()`



groupBy Multiple Aggregations

```
In [3]: sales.groupby('city')[['bread','butter']].max()
Out[3]:
         bread butter
city
Austin     326     70
Dallas     456     98
```

Aggregation functions string names :

- ‘sum’
- ‘mean’
- ‘count’

```
In [4]: sales.groupby('city')[['bread','butter']].agg(['max','sum'])
Out[4]:
       bread     butter
         max sum    max sum
city
Austin   326 465     70  90
Dallas   456 693     98 143
```

original

```
   bread butter   city weekday
0    139     20 Austin     Sun
1    237     45 Dallas     Sun
2    326     70 Austin     Mon
3    456     98 Dallas     Mon
```



Custom Aggregation

original

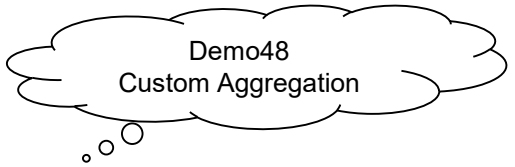
```
bread butter   city weekday
0   139     20 Austin     Sun
1   237     45 Dallas     Sun
2   326     70 Austin     Mon
3   456     98 Dallas     Mon
```

```
In [5]: def data_range(series):
   ...: return series.max() - series.min()

In [6]: sales.groupby('weekday')[['bread', 'butter']].agg(data_range)
Out[6]:
         bread butter
weekday
Mon        130     28
Sun         98     25

In [7]: sales.groupby(customers)[['bread', 'butter']]
   ...: .agg({'bread':'sum', 'butter':data_range})
Out[7]:
         butter bread
Alice        53   693
Bob           0   326
Dave          0   139
```

```
In [9]: customers
Out[9]:
0    Dave
1   Alice
2     Bob
3   Alice
dtype: object
```



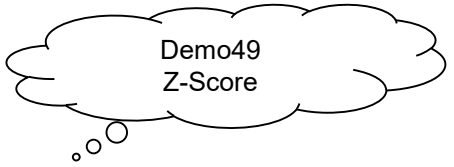
Groupby and Transformation : Automobile Dataset z-score

- **z-score Definition :**
  - Number of standard deviations from the mean a data point is

```
In [2]: auto = pd.read_csv('auto-mpg.csv')
In [3]: auto.head()
Out[3]:
    mpg  cyl      displ     hp      weight accel yr                  origin name
0  18.0    8      307.0    130       3504  12.0 70 US chevrolet chevelle malibu
1  15.0    8      350.0    165       3693  11.5 70         US buick skylark 320
2  18.0    8      318.0    150       3436  11.0 70        US plymouth satellite
3  16.0    8      304.0    150       3433  12.0 70             US amc rebel sst
4  17.0    8      302.0    140       3449  10.5 70               US ford torino
In [1]: def zscore(series):
   ...: return (series - series.mean()) / series.std()
```

- **MPG z-score :**

```
In [4]: zscore(auto['mpg']).head()
Out[4]:
        mpg
0  -0.697747
1  -1.082115
2  -0.697747
3  -0.953992
```

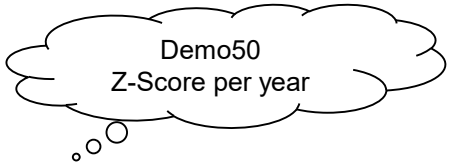


MPG z-score by Year

```
In [5]: auto.groupby('yr')['mpg'].transform(zscore).head()
Out[5]:
          mpg
0    0.058125
1   -0.503753
2    0.058125
3   -0.316460
4   -0.129168
```

Original

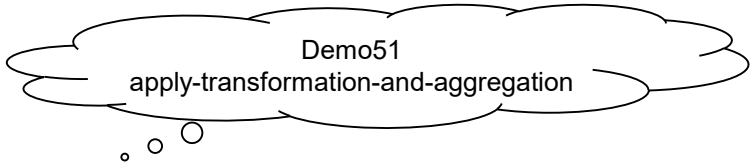
	mpg	cyl	displ	hp	weight	accel	yr	origin	name
0	18.0	8	307.0	130	3504	12.0	70	US	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	US	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	US	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	US	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	US	ford torino



Apply Transformation and Aggregation

```
In [6]: def zscore_with_year_and_name(group):
   ...: df = pd.DataFrame(
   ...: {'mpg': zscore(group['mpg']),
   ...: 'year': group['yr'],
   ...: 'name': group['name']})
   ...: return df

In [7]: auto.groupby('yr').apply(zscore_with_year_and_name).head()
Out[7]:
        mpg                       name year
0  0.058125 chevrolet chevelle malibu   70
1 -0.503753         buick skylark 320   70
2  0.058125        plymouth satellite   70
3 -0.316460             amc rebel sst   70
4 -0.129168               ford torino   70
```

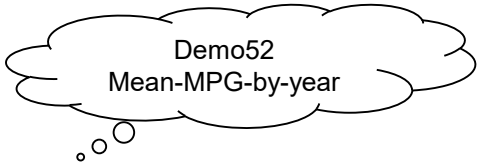


Groupby and Filtering : Mean MPG by Year

```
In [3]: auto.groupby('yr')['mpg'].mean()
Out[3]:
yr
70 17.689655
71 21.111111
72 18.714286
73 17.100000
74 22.769231
75 20.266667
76 21.573529
77 23.375000
78 24.061111
79 25.093103
80 33.803704
81 30.185714
82 32.000000
Name: mpg, dtype: float64
```

Original

	mpg	cyl	displ	hp	weight	accel	yr	origin	name
0	18.0	8	307.0	130	3504	12.0	70	US	Chevrolet malibu
1	15.0	8	350.0	165	3693	11.5	70	US	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	US	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	US	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	US	ford torino

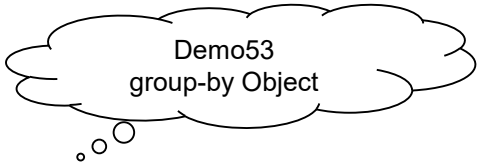


groupby Object

```
In [4]: splitting = auto.groupby('yr')
In [4]: type(splitting)
Out[4]: pandas.core.groupby.DataFrameGroupBy
In [5]: type(splitting.groups)
Out[5]: dict
In [6]: print(splitting.groups.keys())
Out[6]: dict_keys([70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82])
```

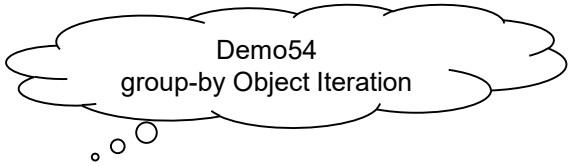
Original

	mpg	cyl	displ	hp	weight	accel	yr	origin	name
0	18.0	8	307.0	130	3504	12.0	70	US	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	US	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	US	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	US	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	US	ford torino



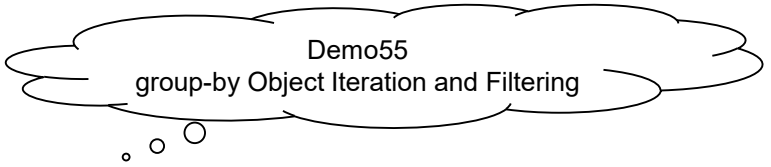
groupby Object : Iteration

```
In [7]: for group_name, group in splitting:
   ...: avg = group['mpg'].mean()
   ...: print(group_name, avg)
Out[7]:
70 17.6896551724
71 21.1111111111
72 18.7142857143
73 17.1
74 22.7692307692
75 20.2666666667
76 21.5735294118
77 23.375
78 24.0611111111
79 25.0931034483
80 33.8037037037
81 30.1857142857
82 32.0
```



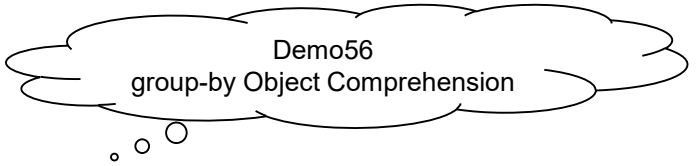
groupby Object: Iteration and Filtering

```
In [8]: for group_name, group in splitting:
   ...: avg = group.loc[group['name'].str.contains('chevrolet'), 'mpg'].mean()
   ...: print(group_name, avg)
Out[8]:
70 15.6666666667
71 20.25
72 15.3333333333
73 14.8333333333
74 18.6666666667
75 17.6666666667
76 23.25
77 20.25
78 23.2333333333
79 21.6666666667
80 30.05
81 23.5
82 29.0
```



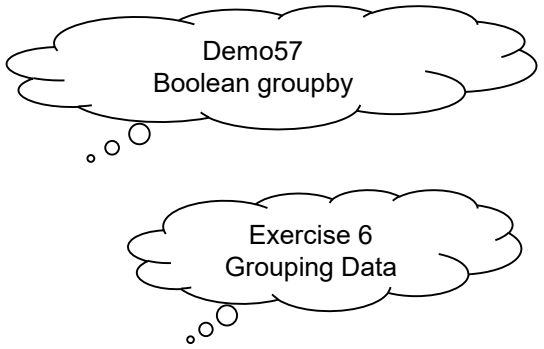
groupby Object : Comprehension

```
In [9]: chevy_means =
{year:group.loc[group['name'].str.contains('chevrolet'),'mpg'].mean()
   ...: for year,group in splitting}
In [10]: pd.Series(chevy_means)
Out[10]:
70 15.666667
71 20.250000
72 15.333333
73 14.833333
74 18.666667
75 17.666667
76 23.250000
77 20.250000
78 23.233333
79 21.666667
80 30.050000
81 23.500000
82 29.000000
dtype: float64
```



Boolean groupby

```
In [11]: chevy = auto['name'].str.contains('chevrolet')
In [12]: auto.groupby(['yr', chevy])['mpg'].mean()
Out[12]:
yr name
70 False   17.923077
   True    15.666667
71 False   21.260870
   True    20.250000
72 False   19.120000
   True    15.333333
73 False   17.500000
   True    14.833333
74 False   23.304348
   True    18.666667
75 False   20.555556
   True    17.666667
76 False   21.350000
   True    23.250000
```



SpiralTrain
developer courses

Course Schedule

- Numpy
- Pandas Intro
- Extracting and Transforming Data
- Advanced Indexing
- Rearranging Data
- Grouping Data
- **MatplotLib**
- Time Series

- **What is MatplotLib?**
- **MatplotLib Objects**
- **Using MatplotLib**
- **Input Types**
- **Single Values Plot**
- **Two Values Plot**
- **Statefullness**
- **Format Strings**
- **Multiple Plots**
- **Line Properties**
- **Labels and Legenda**
- **Styling**
- **Text in MatplotLib**
- **Text Properties**
- **Text and Annotations**
- **Scatter Plots**
- **Sub Plots**
- **Datetime Plots**
- **Transform Datetime**

© copyright : spiraltrain@gmail.com

What is MatplotLib?

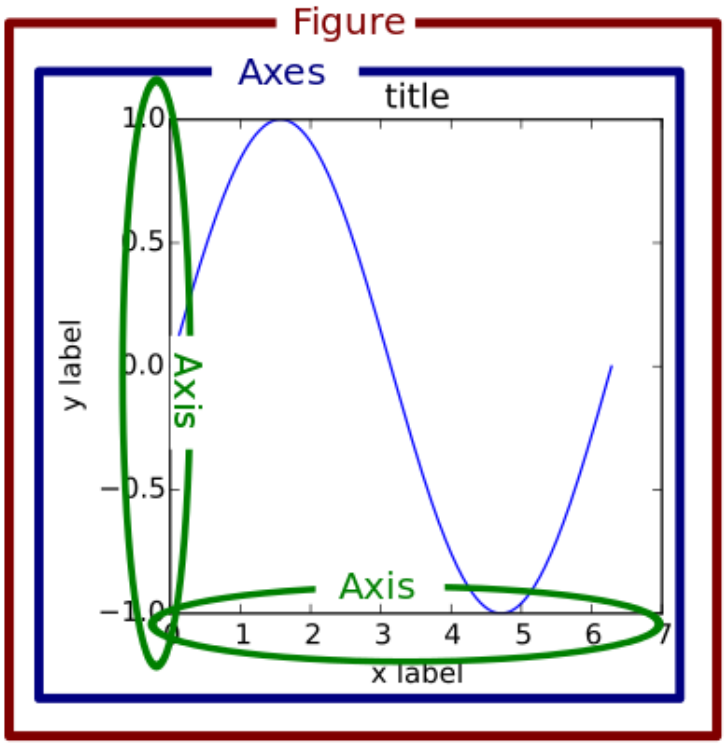
- **Python Plotting Library at** `https://matplotlib.org/` **:**
  - Open source 2D-plot library producing visually appealing plots
- **Useable in :**
  - Python scripts and interactive environments like iPython notebook and IPython Shell
- **Directly embeddable into web applications :**
  - And also four graphical user interface toolkits
- **Written in Python, but heavily using NumPy :**
  - Shows good performance even on large data structures
- **At high level in matplotlib is** `matplotlib.pyplot` **module :**
  - Simple functions are used to add plot lines, images, text, etc. to current figure
- **Next level down is level of object-oriented interface :**
  - `pyplot` is used only to explicitly create figure and axes objects
- **For things like embedding matplotlib plots in GUI applications :**
  - `pyplot` level may be dropped leaving purely object-oriented approach

MatplotLib Objects

- `Figure` **object represents whole figure :**
  - Figure keeps track of child Axes, artist like titles, legends, text and the canvas
- `Figure` **can have any number of** `Axes` **:**
  - To be useful `Figure` should have at least one `Axes` object
- **Create empty figure with no axes with** `fig = plt.figure()` **:**
  - `Figure` with a 2x2 grid of `Axes` : `fig, ax_lst = plt.subplots(2, 2)`
- `Axes` **object is what you think of as a plot :**
  - Given figure can contain many `Axes`, but `Axes` object can only be in one `Figure`
- **Each** `Axes` **has** `title`, `x-label` **and** `y-label` **:**
  - Set via `set_title()`, `set_xlabel()` and `set_ylabel()`
- `Axes` **typically contains two** `Axis` **objects :**
  - Be aware of the difference between `Axes` and `Axis`
- `Axis` **objects set graph limits and marks on the axis :**
  - `Locator` and `Formatter` objects give fine control over tick locations and labels

Axes versus Axis

- Axis is the axis of the plot that gets ticks and tick labels
- Axes is the area your plot appears in



Using MatplotLib

- **Usual import statement to use matplotlib :**
  - `from matplotlib import pyplot as plt`
  - `%matplotlib inline`
- `pyplot` **:**
  - Collection of tools that provide MATLAB-like plotting functionality
- `%matplotlib inline` **:**
  - Resulting plots should be embedded directly into ipython notebook
  - Instead of starting an external graphical user interface
- `pyplot` **provides various functions to modify figures :**
  - plot values, change labels, change titles, add legends, etc
  - Plotting functions are directed to the current axes
- `pyplot` **is stateful :**
  - Keeps track of all changes to current figure and plotting area
- `pyplot` **gives state-machine interface to object oriented plot library :**
  - Automatically creates figures and axes to achieve desired plot

Input Types

- **Plot functions expect `np.array` or `np.ma.masked_array` as input :**
  - Array-like classes such as pandas data objects and `np.matrix` may not work
  - Best to convert these to `np.array` objects prior to plotting
- **Convert `pandas.DataFrame` as follows :**

```
a = pandas.DataFrame(np.random.rand(4,5), columns = list('abcde'))
a_asndarray = a.values
```

- **Convert `np.matrix` as follows :**

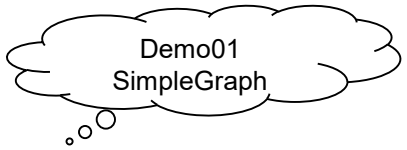
```
b = np.matrix([[1,2],[3,4]])
b_asarray = np.asarray(b)
```

- **Plot functions can also work with Python lists :**

```
from matplotlib import pyplot as plt
# Plotting to our canvas
plt.plot([1, 2, 3, 4, 5 ], [2, -4, 6, -8, 10])
```

- **In `IPython` shell plot is automatically displayed :**
  - Otherwise we can show what we plotted with :

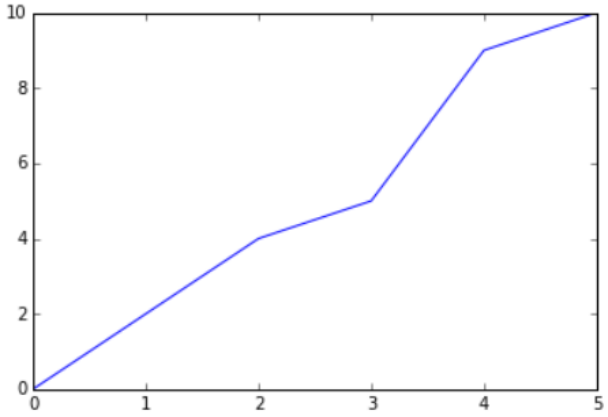
```
plt.show()
```



Single Values Plot

- `pyplot` **assumes single list of values are** `y` **values :**
  - Automatically generates x values by using `range(len(values))`

```
from matplotlib import pyplot as plt
import numpy as np
values = np.array([0,2,4,5,9,10])
plt.plot(values)
```

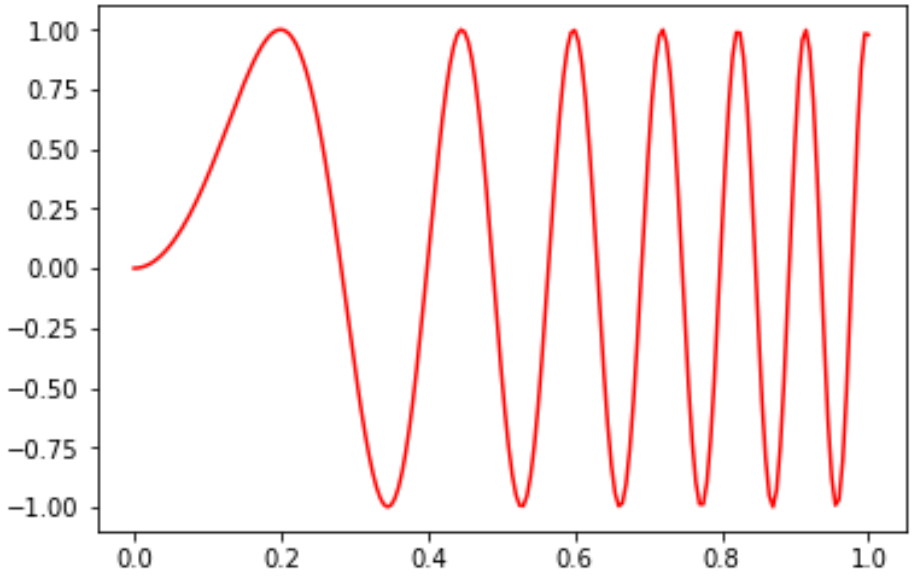


- **In absence of an x values list :**
  - `[0, 1, 2, 3, 4, 5 ]` was used instead

Demo02
Single Values Plot

Line Plots

```
from matplotlib import pyplot as plt
import numpy as np
x = np.linspace(0, 1, 201)
y = np.sin((2*np.pi*x)**2)
plt.plot(x, y, 'red')
plt.show()
```

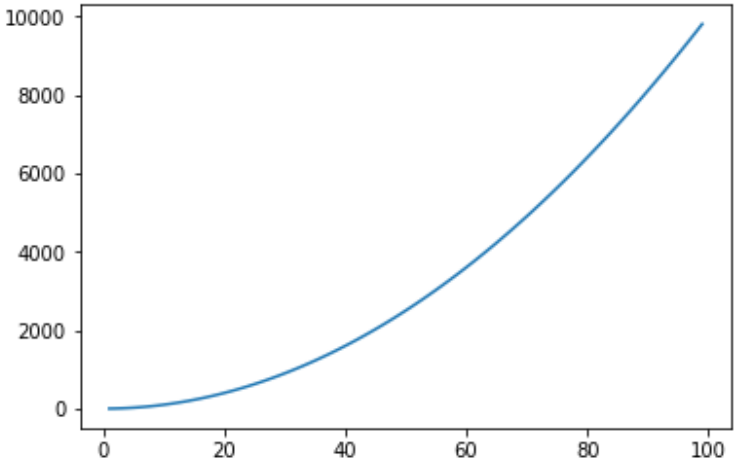


Demo03
Line Plots

Two Values Plot

- **To plot `x` versus `y` :**
  - Need two provide two sequences of values
  - First sequence will be interpreted as `x` and second one as `y` values

```
from matplotlib import pyplot as plt
import numpy as np
values = range(1,100)
plt.plot(values, np.square(values))
plt.show()
```



Demo04
Two Values Plot

Statefulness

- `pyplot` **is stateful :**
  - Can either call `plot` several times and each call adds to the figure
- **First call to** `plt.plot` **:**
  - Automatically creates necessary figure and axes to achieve desired plot
- **Subsequent calls to** `plt.plot` **re-use current axes :**
  - Setting title, legend, and axis labels also automatically use current axes
- **Plot function of** `matplotlib` **has following interface :**
  - `plot(*args, **kwargs)`
- **Optional third argument of** `plot` **function :**
  - Interpreted as a format string
- **Can provide multiple pairs of** `x,y` **format string pairs :**

```
plot(x, y)         # plot x and y using default line style and color
plot(x, y, 'bo')   # plot x and y using blue circle markers
plot(y)            # plot y using x as index array 0..N-1
plot(y, 'r+')      # ditto, but with red plusses
```

Format Strings

- **There are plenty of format strings :**
  - List of all available format strings can be found in pyplot documentation

`https://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.plot`

- **Marker styles**

character	description
'-'	solid line style
'--'	dashed line style
'-.'	dash-dot line style
':'	dotted line style
'.'	point marker
','	pixel marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker

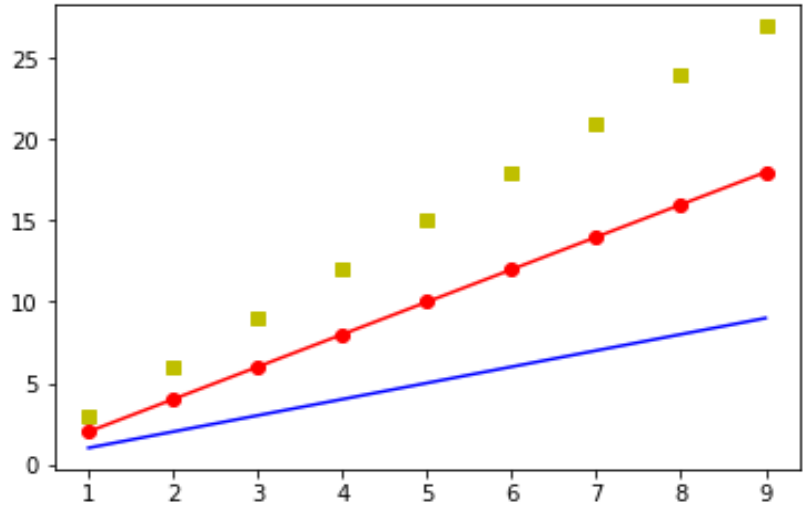
Line colors

character	color
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

Multiple Plots

- **Default is `b-` meaning connect entries by blue colored solid line**

```
values = list(range(1, 10))
values2 = [x*2 for x in values ]
values3 = [x*3 for x in values ]
plt.plot(values, values, 'b-') #default "b-"
plt.plot(values, values2, 'ro-’)
plt.plot(values, values3, 'ys’)
plt.show()
```



Demo05
Multiple Plots

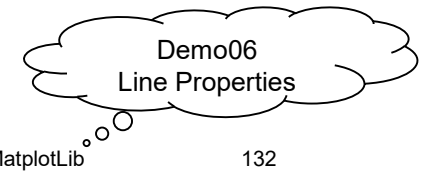
Line Properties

- **Line attributes like `linewidth`, `color` etc. can be changed :**
  - Using either keyword arguments to the `plot` function
  - Using the set properties `setp()` function

```
plt.plot(values, values, 'b-', linewidth=5.0)
myplot = plt.plot(values, values2, 'b-')
myplot = plt.setp(myplot, color='r')
```

- **List of available line properties found in `pyplot` documentation**
  - `https://matplotlib.org/api/_as_gen/matplotlib.lines.Line2D.html`

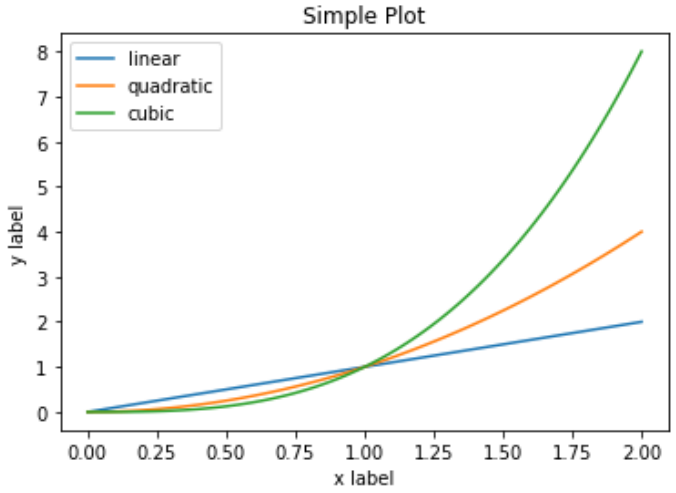
dash_capstyle	{'butt', 'round', 'projecting'}
dash_joinstyle	{'miter', 'round', 'bevel'}
dashes	sequence of floats (on/off ink in points) or (None, None)
drawstyle	{'default', 'steps', 'steps-pre', 'steps-mid', 'steps-post'}
figure	Figure
fillstyle	{'full', 'left', 'right', 'bottom', 'top', 'none'}



Labels and Legenda

- **Setting title, legend, and axis labels done as follows :**

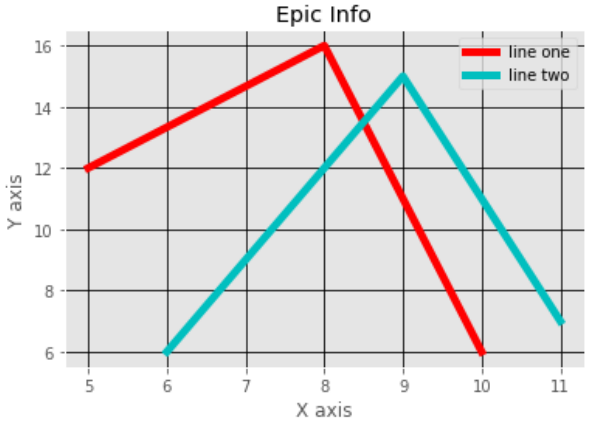
```
x = np.linspace(0, 2, 100)
plt.plot(x, x, label='linear')
plt.plot(x, x**2, label='quadratic')
plt.plot(x, x**3, label='cubic')
plt.xlabel('x label')
plt.ylabel('y label')
plt.title("Simple Plot")
plt.legend()
```



Demo07
Adding Legenda

Styling

- `style` **package adds support for easy-to-switch plotting styles :**
  - With same parameters as in matplotlib configuration file matplotlib rc
- **There are number of pre-defined styles provided by Matplotlib :**
  - Pre-defined style `ggplot` emulates aesthetics of popular plotting package for R
- **Other styles are :**
  - `seaborn-dark-palette, grayscale, classic` and more
- **Can create custom styles and use them by calling** `style.use` **:**
  - With the path or URL to the style sheet



Demo08
Styling

Text in MatplotLib

- **Matplotlib has extensive text support :**
  - Mathematical expressions, newline separated text with arbitrary rotations and Unicode
- **Great deal of control over text properties with sensible defaults set :**
  - font size, font weight, text location and color
- **Matplotlib implements large number of TeX math symbols:**
  - Supporting mathematical expressions anywhere in your figure
- `text()` **function used to place text at an arbitrary position :**
  - `LaTeX` can be used as well
- **Also possible to surround text by bounding box :**
  - Just add the `bbox` keyword argument to `text()` call
- **Mark specific points in plot using arrows :**
  - Can use `annotate` function
- **Both text() and annotate have many properties to adjust :**
  - Find details in `pyplot` documentation

`http://matplotlib.org/users/annotations_guide.html`

Text Properties

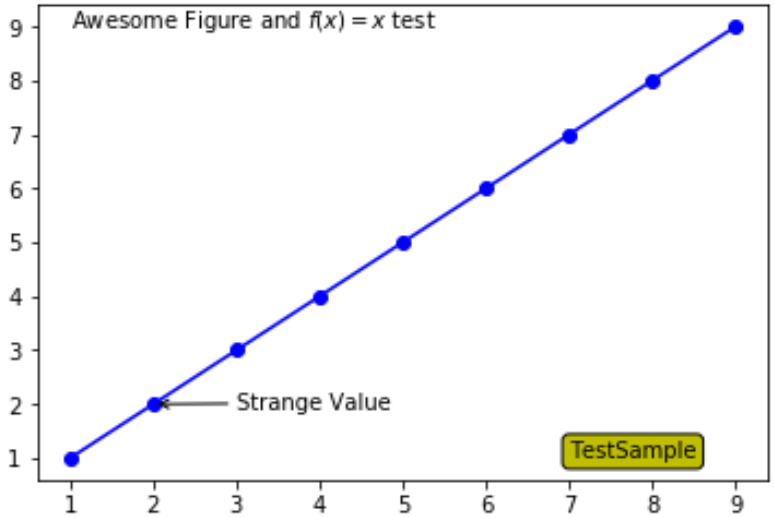
- **Text properties can be changed by :**
  - Either keyword arguments or using `setp()` function
- **Further text properties and layouts in `pyplot` documentation :**
  `https://matplotlib.org/users/text_props.html`

Property	Value Type
alpha	`float`
backgroundcolor	any matplotlib color
bbox	`Rectangle` prop dict plus key `'pad'` which is a pad in points
clip_box	a matplotlib.transform.Bbox instance
clip_on	[True | False]
clip_path	a `Path` instance and a `Transform` instance, a `Patch`
color	any matplotlib color
family	[`'serif'` | `'sans-serif'` | `'cursive'` | `'fantasy'` | `'monospace'`]
fontproperties	a `FontProperties` instance
horizontalalignment or	[`'center'` | `'right'` | `'left'`]

Text and Annotations

- **Add text and annotations to figure :**

```
from matplotlib import pyplot as plt
values = list(range(1, 10))
plt.plot(values, values, 'bo-’)
plt.text(1, 9, 'Awesome Figure and f(x)=x test')# x, y values plus text
plt.annotate('Strange Value', xy=(2, 2),
             xytext=(3, 1.9),
             arrowprops=dict(facecolor='black', arrowstyle="->"))
plt.text(7, 1, "TestSample", bbox=dict(boxstyle="round", fc='y'))
```



Demo09
Text Annotations

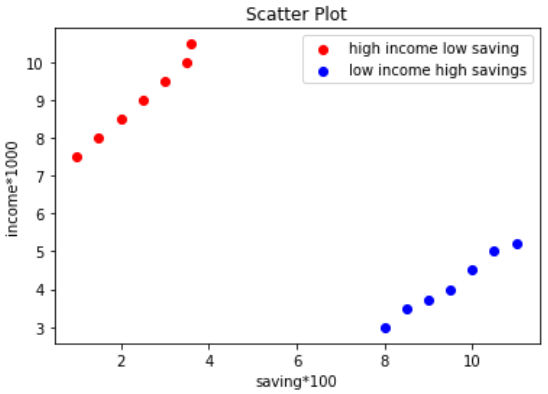
Scatter Plots

- **Two-dimensional data visualization using dots for values :**
  - Values obtained for two different variables
  - One plotted along the x-axis and the other plotted along the y-axis

```
import matplotlib.pyplot as plt

x = [1, 1.5, 2, 2.5, 3, 3.5, 3.6]
y = [7.5, 8, 8.5, 9, 9.5, 10, 10.5]

x1 = [8, 8.5, 9, 9.5, 10, 10.5, 11]
y1 = [3, 3.5, 3.7, 4, 4.5, 5, 5.2]
```



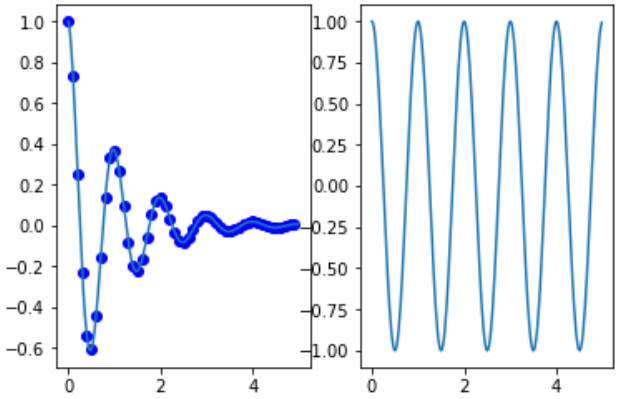

```
plt.scatter(x, y, label='high income low saving', color='r')
plt.scatter(x1, y1, label='low income high savings', color='b')
plt.xlabel('saving*100')
plt.ylabel('income*1000')
plt.title('Scatter Plot')
plt.legend()
plt.show()
```

Demo10
Scatter Plot

Sub Plots

- `pyplot` **can deal with several figures and subplots :**
  - Multiple plots can be plotted besides each other
- `subplot(nrows, ncols, nsubplot)`
- **Subplot ordering :**
  - Row-wise from top left
  - Indexed from 1

```
def f(t):
    return np.exp(-t) * np.cos(2 * np.pi * t)
t1 = np.arange(0.0, 5.0, 0.1)
t2 = np.arange(0.0, 5.0, 0.02)
plt.subplot(121)
plt.plot(t1, f(t1), 'bo', t2, f(t2))
plt.subplot(122)
plt.plot(t2, np.cos(2 * np.pi * t2))
plt.show()
```



Demo11
Sub Plots

Datetime Plots

- **Sometimes it is necessary to plot values versus datetime :**
  - `pyplot` provides possibility to create datetime plots

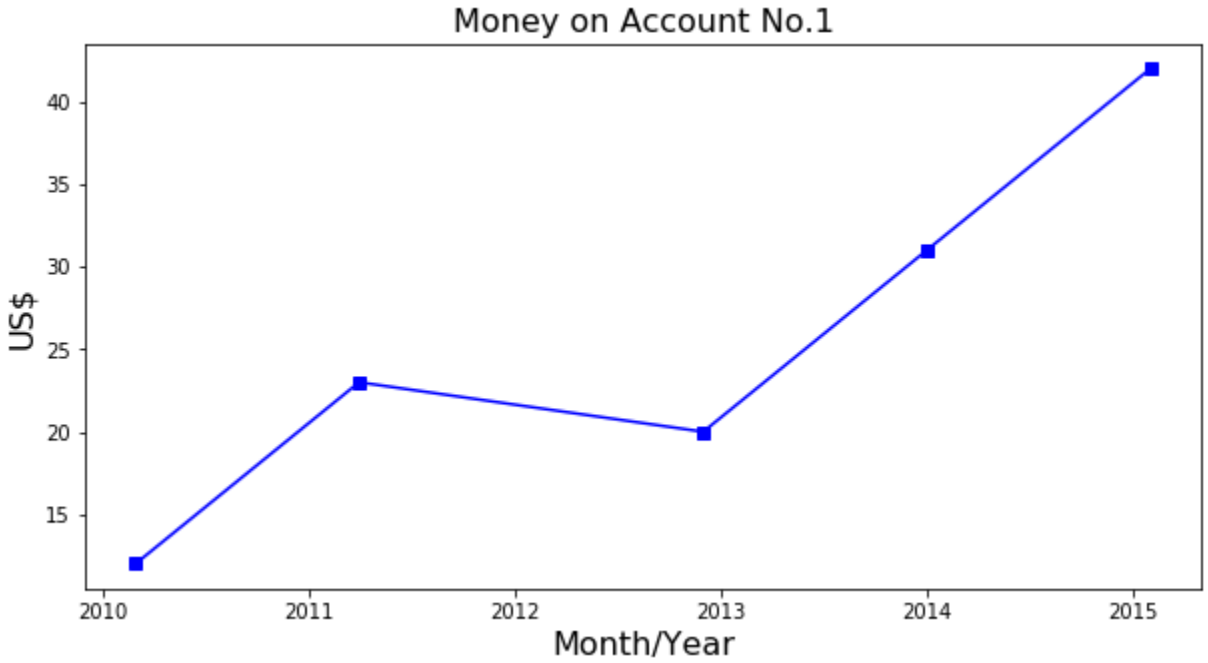
```
import matplotlib.pyplot as plt

date_values = {'date': ['2010/03', '2011/04', '2012/12', '2014/01', '2015/02'],
               'value': [12, 23, 20, 31, 42]}

#transform date from string into datetime format. This is important!
from datetime import datetime
date_values['date'] =
   list(map(lambda x: datetime.strptime(x, '%Y/%m'), date_values['date']))
fig = plt.figure(figsize=(10,5))
plt.plot_date(date_values['date'], date_values['value'], 'bs-')
plt.title('Money on Account No.1', fontsize=16)
plt.xlabel('Month/Year', fontsize=16)
plt.ylabel('US$', fontsize=16)
```

Transform Datetime

- **If `date` values are initially provided as string :**
  - Have to transform datetime string into a python `datetime` object!
  - More information can be found in pyplot documentation

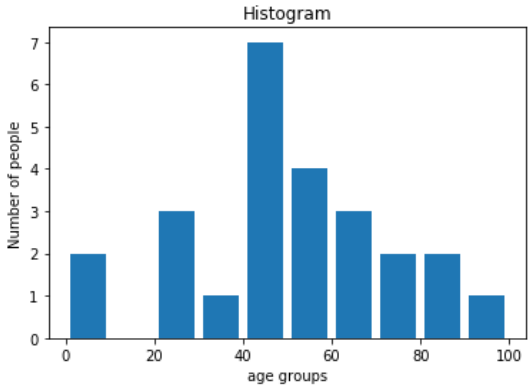


Demo12
Datetimeplots

Histograms

- `pylot` **is able to compute and draw histograms :**
  - Input data is passed as `NumPy` array to `hist()` function of pyplot
- **Binning can be adjusted :**
  - Using `bins` keyword argument in `hist()` function call
- **Several parameters can adjust properties of histogram**
  - Please, have a look at the pyplot documentation.

```
import numpy as np
gausian_sim_data = np.random.normal(size=10000)
plt.hist(gausian_sim_data, bins=20)
```



- **2D-histograms are available as well :**
  - Just call `hist2d()` instead of `hist()`
  - More information in pyplot documentation

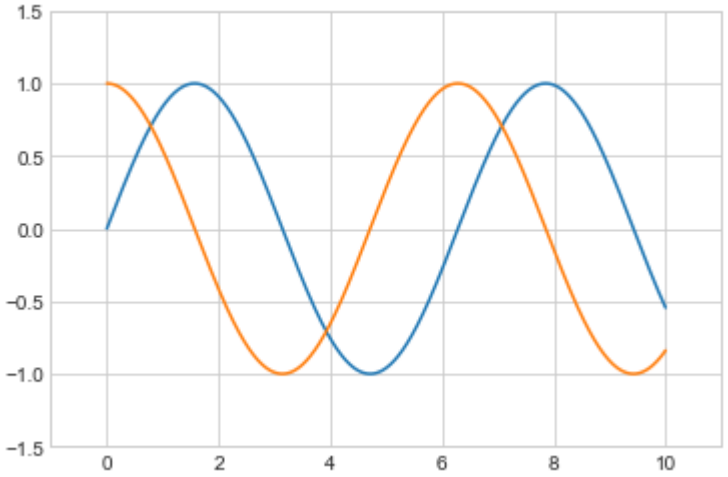
Demo13
Histograms

Using axes()

```
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np

fig = plt.figure()
ax = plt.axes()

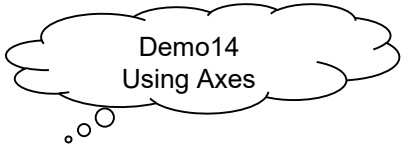
x = np.linspace(0, 10, 1000)
ax.plot(x, np.sin(x));
ax.plot(x, np.cos(x));
```



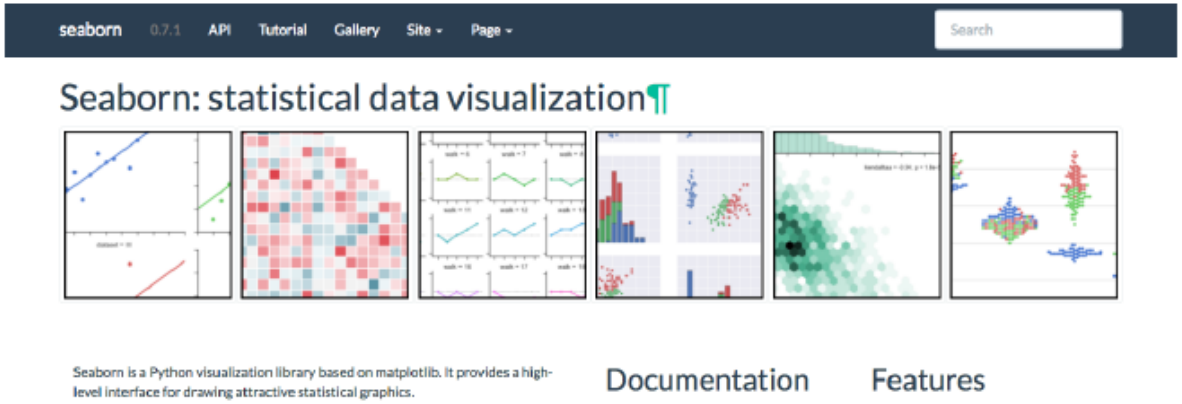
- **Control over individual axis extents :**
  - `xlim([xmin, xmax])`
  - `ylim([ymin, ymax])`

```
plt.xlim(-1, 11)
plt.ylim(-1.5, 1.5);
```

- `plt.axis()` **allows you to set** `x` **and** `y` **limits in single call :**
  - By passing a list which specifies `[xmin, xmax, ymin, ymax]`
- **Controlling axis extents :**
  - `axis([xmin, xmax, ymin, ymax])` sets axis extents



Seaborn



https://stanford.edu/~mwaskom/software/seaborn/

http://seaborn.pydata.org/tutorial.html

Demo15
Piechart

Demo16
Barchart

Demo17
AreaPlot

Demo18
PlotTangens

Demo19
PlotSinus

© copyright : spiraltrain@gmail.com

Course Schedule

- Numpy
- Pandas Intro
- Extracting and Transforming Data
- Advanced Indexing
- Rearranging Data
- Grouping Data
- MatplotLib
- **Time Series**

- **Indexing time series**
- **Parse dates**
- **Selecting Dates**
- **Selecting whole day**
- **Partial datetime selection**
- **Selecting whole month**
- **Reindex DataFrame**
- **Filling Missing Values**
- **Resampling Data**
- **Sales Data**
- **Manipulating time series**
- **Multiplying Frequencies**
- **Upsampling and Filling**
- **Set Time Zone**
- **Convert Time Zone**
- **Method Chaining**
- **UpSampling**

© copyright : spiraltrain@gmail.com

Read datetime Objects

- `read_csv()` **function can read strings into datetime objects :**
  - Need to specify `parse_dates=True` or specifically name the columns

```
sales = pd.read_csv("sales-feb-2015.csv", parse_dates=['Date'], index_col= "Date")
```

- **ISO 8601 format :**
  - `yyyy-mm-dd hh:mm:ss`
- **Consider Product sales CSV :**

	Date	Company	Product	Units
0	2015-02-02 08:30:00	Hooli	Software	3
1	2015-02-02 21:00:00	Mediacore	Hardware	9
2	2015-02-03 14:00:00	Initech	Software	13
3	2015-02-04 15:30:00	Streeplex	Software	13
4	2015-02-04 22:00:00	Acme Coporation	Hardware	14

Parsing Dates

```
In [1]: import pandas as pd
In [2]: sales = pd.read_csv('sales-feb-2015.csv',
   ...: parse_dates=True, index_col= 'Date')
In [3]: sales.head()
Out[3]:
                                Company          Product           Units
Date
2015-02-02 08:30:00                Hooli         Software             3
2015-02-02 21:00:00            Mediacore         Hardware             9
2015-02-03 14:00:00              Initech         Software            13
2015-02-04 15:30:00             Streeplex        Software            13
2015-02-04 22:00:00         Acme Coporation      Hardware            14
In [4]: sales.info()
DatetimeIndex: 19 entries, 2015-02-02 08:30:00 to 2015-02-26 09:00:00
Data columns (total 3 columns):
Company 19 non-null object
Product 19 non-null object
Units 19 non-null int64
dtypes: int64(1), object(2)
memory usage: 608.0+ bytes
```

Selecting Dates

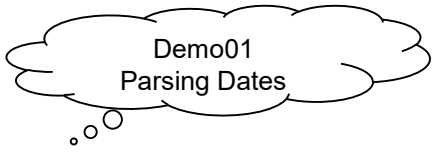
- **Selecting single datetime :**

```
In [5]: sales.loc['2015-02-19 11:00:00', 'Company']
Out[5]: 'Mediacore'
```

- **Selecting whole day :**

```
In [6]: sales.loc['2015-2-5']
Out[6]:
Company Product Units
Date
2015-02-05 02:00:00 Acme Coporation Software 19
2015-02-05 22:00:00 Hooli Service 10
```

- **Partial datetime string selection :**
  - Alternative formats :
    - `sales.loc['February 5, 2015']`
    - `sales.loc['2015-Feb-5']`
  - Whole month : `sales.loc['2015-2']`
  - Whole year : `sales.loc['2015']`



Selecting Whole Month

```
In [7]: sales.loc['2015-2']
Out[7]:
```

Date	Company	Product	Units
2015-02-02 08:30:00	Hooli	Software	3
2015-02-02 21:00:00	Mediacore	Hardware	9
2015-02-03 14:00:00	Initech	Software	13
2015-02-04 15:30:00	Streeplex	Software	13
2015-02-04 22:00:00	Acme Coporation	Hardware	14
2015-02-05 02:00:00	Acme Coporation	Software	19
2015-02-05 22:00:00	Hooli	Service	10
2015-02-07 23:00:00	Acme Coporation	Hardware	1
2015-02-09 09:00:00	Streeplex	Service	19
2015-02-09 13:00:00	Mediacore	Software	7
2015-02-11 20:00:00	Initech	Software	7
2015-02-11 23:00:00	Hooli	Software	4
2015-02-16 12:00:00	Hooli	Software	10
2015-02-19 11:00:00	Mediacore	Hardware	16

Slicing and Converting

- **Slicing using Dates and Times :**

```
In [8]: sales.loc['2015-2-16':'2015-2-20']
Out[8]:
                                Company          Product          Units
Date
2015-02-16 12:00:00               Hooli         Software             10
2015-02-19 11:00:00           Mediacore         Hardware             16
2015-02-19 16:00:00           Mediacore          Service             10
```

- **Convert Strings to datetime :**

```
In [9]: evening_2_11 = pd.to_datetime(['2015-2-11 20:00',
   ...: '2015-2-11 21:00', '2015-2-11 22:00', '2015-2-11 23:00'])
In [10]: evening_2_11
Out[10]:
DatetimeIndex(['2015-02-11 20:00:00', '2015-02-11 21:00:00',
               '2015-02-11 22:00:00', '2015-02-11 23:00:00'],
               dtype='datetime64[ns]', freq=None)
```

Reindexing and Filling

```
In [11]: sales.reindex(evening_2_11)
Out[11]:
                                Company          Product          Units
2015-02-11 20:00:00             Initech          Software           7.0
2015-02-11 21:00:00                 NaN              NaN           NaN
2015-02-11 22:00:00                 NaN              NaN           NaN
2015-02-11 23:00:00               Hooli         Software           4.0
```

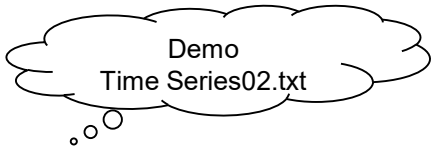
- **Filling missing data**

```
In [12]: sales.reindex(evening_2_11, method='ffill')
Out[12]:
                                Company          Product          Units
2015-02-11 20:00:00             Initech          Software             7
2015-02-11 21:00:00             Initech          Software             7
2015-02-11 22:00:00             Initech          Software             7
2015-02-11 23:00:00               Hooli          Software             4
In [13]: sales.reindex(evening_2_11, method='bfill')
Out[13]:
                                Company          Product          Units
2015-02-11 20:00:00             Initech          Software             7
2015-02-11 21:00:00               Hooli          Software             4
2015-02-11 22:00:00               Hooli          Software             4
2015-02-11 23:00:00               Hooli          Software             4
```

Resampling Sales Data

```
In [1]: import pandas as pd
In [2]: sales = pd.read_csv('sales-feb-2015.csv',
   ...: parse_dates=True, index_col= 'Date')
In [3]: sales.head()
Out[3]:
                                Company         Product          Units
Date
2015-02-02 08:30:00               Hooli        Software              3
2015-02-02 21:00:00           Mediacore        Hardware              9
2015-02-03 14:00:00             Initech        Software             13
2015-02-04 15:30:00           Streeplex        Software             13
2015-02-04 22:00:00    Acme Coporation         Hardware             14
```

- **Statistical methods over different time intervals :**
  - `mean()`, `sum()`, `count()`, etc.
- **Down-sampling :**
  - Reduce datetime rows to slower frequency
- **Up-sampling :**
  - Increase datetime rows to faster frequency



Resampling Frequencies

Input	Description
'min', ' T'	minute
'H'	hour
'D'	day
'B'	business day
'W'	week
'M'	month
'Q'	quarter
'A'	year

Aggregating Means

- **Parameter value D means resample per day**

```
In [4]: daily_mean = sales.resample('D').mean()
In [5]: daily_mean
Out[5]:
              Units
Date
2015-02-02     6.0
2015-02-03    13.0
2015-02-04    13.5
2015-02-05    14.5
2015-02-06     NaN
2015-02-07     1.0
2015-02-08     NaN
2015-02-09    13.0
2015-02-10     NaN
2015-02-11     5.5
2015-02-12     NaN
2015-02-13     NaN
```

Verifying Results

```
In [6]: print(daily_mean.loc['2015-2-2'])
Units 6.0
Name: 2015-02-02 00:00:00, dtype: float64
In [7]: print(sales.loc['2015-2-2', 'Units'])
Date
2015-02-02 08:30:00 3
2015-02-02 21:00:00 9
Name: Units, dtype: int64
In [8]: sales.loc['2015-2-2', 'Units'].mean()
Out[8]: 6.0
```

Method Chaining

```
In [9]: sales.resample('D').sum()
Out[9]:
            Units
Date
2015-02-02   6.0
2015-02-03  13.0
2015-02-04  13.5
2015-02-05  14.5
2015-02-06   NaN
2015-02-07   1.0
2015-02-08   NaN
2015-02-09  13.0
2015-02-10   NaN
2015-02-11   5.5
2015-02-12   NaN
2015-02-13   NaN
```

```
In [10]: sales.resample('D').sum().max()
Out[10]:
Units 29.0
dtype: float64
```

Resampling by Week

```
In [11]: sales.resample('W').count()
Out[11]:
                Company          Product          Units
Date
2015-02-08              8                8              8
2015-02-15              4                4              4
2015-02-22              5                5              5
2015-03-01              2                2              2
```

- **Multiplying Frequencies**

```
In [12]: sales.loc[:,'Units'].resample('2W').sum()
Out[12]:
Date
2015-02-08 82
2015-02-22 79
2015-03-08 14
Freq: 2W-SUN, Name: Units, dtype: int64
```

Upsampling

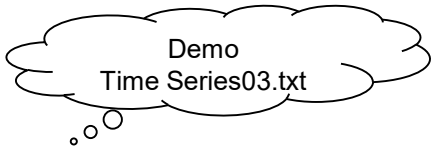
```
In [13]: two_days = sales.loc['2015-2-4': '2015-2-5', 'Units']
In [13]: two_days
Out[13]:
Date
2015-02-04 15:30:00 13
2015-02-04 22:00:00 14
2015-02-05 02:00:00 19
2015-02-05 22:00:00 10
Name: Units, dtype: int64
```

Upsampling and filling

```
In [14]: two_days.resample('4H').ffill()
Out[14]:
Date
Date
2015-02-04 12:00:00 NaN
2015-02-04 16:00:00 13.0
2015-02-04 20:00:00 13.0
2015-02-05 00:00:00 14.0
2015-02-05 04:00:00 19.0
2015-02-05 08:00:00 19.0
2015-02-05 12:00:00 19.0
2015-02-05 16:00:00 19.0
2015-02-05 20:00:00 19.0
Freq: 4H, Name: Units, dtype: float64
```

Sales data

```
In [1]: import pandas as pd
In [2]: sales = pd.read_csv('sales-feb-2015.csv',
   ...: parse_dates=['Date'])
In [3]: sales.head()
Out[3]:
                  Date              Company          Product  Units
0 2015-02-02 08:30:00                Hooli         Software      3
1 2015-02-02 21:00:00            Mediacore         Hardware      9
2 2015-02-03 14:00:00              Initech         Software     13
3 2015-02-04 15:30:00            Streeplex         Software     13
4 2015-02-04 22:00:00      Acme Coporation         Hardware     14
```



String methods

```
In [4]: sales['Company'].str.upper()
Out[4]:
0                 HOOLI
1             MEDIACORE
2               INITECH
3             STREEPLEX
4        ACME COPORATION
5        ACME COPORATION
6                 HOOLI
7        ACME COPORATION
8             STREEPLEX
9             MEDIACORE
10              INITECH
11                HOOLI
12                HOOLI
13            MEDIACORE
14            MEDIACORE
15            MEDIACORE
```

Substring matching

```
In [5]: sales['Product'].str.contains('ware')
Out[5]:
0        True
1        True
2        True
3        True
4        True
5        True
6       False
7        True
8       False
9        True
10       True
11       True
12       True
13       True
14      False
...
```

Boolean arithmetic

```
In [6]: True + False
Out[6]: 1
In [7]: True + True
Out[7]: 2
In [8]: False + False
Out[8]: 0
```

- **Boolean reduction**

```
In [9]: sales['Product'].str.contains('ware').sum()
Out[9]: 14
```

Datetime methods

```
In [9]: sales['Date'].dt.hour
Out[9]:
0        8
1       21
2       14
3       15
4       22
5        2
6       22
7       23
8        9
9       13
10      20
11      23
12      12
13      11
14      16
```

Set timezone

```
In [10]: central = sales['Date'].dt.tz_localize('US/Central')
In [11]: central
Out[11]:
0 2015-02-02 08:30:00-06:00
1 2015-02-02 21:00:00-06:00
2 2015-02-03 14:00:00-06:00
3 2015-02-04 15:30:00-06:00
4 2015-02-04 22:00:00-06:00
5 2015-02-05 02:00:00-06:00
6 2015-02-05 22:00:00-06:00
7 2015-02-07 23:00:00-06:00
8 2015-02-09 09:00:00-06:00
9 2015-02-09 13:00:00-06:00
10 2015-02-11 20:00:00-06:00
11 2015-02-11 23:00:00-06:00
12 2015-02-16 12:00:00-06:00
…
Name: Date, dtype: datetime64[ns, US/Central]
```

Convert timezone

```
In [12]: central.dt.tz_convert('US/Eastern')
Out[12]:
0 2015-02-02 09:30:00-05:00
1 2015-02-02 22:00:00-05:00
2 2015-02-03 15:00:00-05:00
3 2015-02-04 16:30:00-05:00
4 2015-02-04 23:00:00-05:00
5 2015-02-05 03:00:00-05:00
6 2015-02-05 23:00:00-05:00
7 2015-02-08 00:00:00-05:00
8 2015-02-09 10:00:00-05:00
9 2015-02-09 14:00:00-05:00
10 2015-02-11 21:00:00-05:00
11 2015-02-12 00:00:00-05:00
12 2015-02-16 13:00:00-05:00
13 2015-02-19 12:00:00-05:00
14 2015-02-19 17:00:00-05:00
…
Name: Date, dtype: datetime64[ns, US/Eastern]
```

Method chaining

```
In [13]: sales['Date'].dt.tz_localize('US/Central').
    ...: dt.tz_convert('US/Eastern')

Out[13]:
0 2015-02-02 09:30:00-05:00
1 2015-02-02 22:00:00-05:00
2 2015-02-03 15:00:00-05:00
3 2015-02-04 16:30:00-05:00
4 2015-02-04 23:00:00-05:00
5 2015-02-05 03:00:00-05:00
6 2015-02-05 23:00:00-05:00
7 2015-02-08 00:00:00-05:00
8 2015-02-09 10:00:00-05:00
9 2015-02-09 14:00:00-05:00
10 2015-02-11 21:00:00-05:00
11 2015-02-12 00:00:00-05:00
12 2015-02-16 13:00:00-05:00
13 2015-02-19 12:00:00-05:00
14 2015-02-19 17:00:00-05:00
…
Name: Date, dtype: datetime64[ns, US/Eastern]
```

SpiralTrain
developer courses

Course Schedule

- Numpy
- Pandas Intro
- Extracting and Transforming Data
- Advanced Indexing
- Rearranging Data
- Grouping Data
- MatplotLib
- Time Series
- **Extra**

© copyright : spiraltrain@gmail.com

Reading Multiple Data Files

- **Tools for pandas data import**
- **pd.read_csv() for CSV files :**
  - dataframe = pd.read_csv(filepath)
  - dozens of optional input parameters
- **Other data import tools :**
  - pd.read_excel()
  - pd.read_html()
  - pd.read_json()
- **Loading separate files :**

```
In [1]: import pandas as pd
In [2]: dataframe0 = pd.read_csv('sales-jan-2015.csv')
In [3]: dataframe1 = pd.read_csv('sales-feb-2015.csv')
```

Reading Multiple Files

- **Using a Loop :**

```
In [4]: filenames = ['sales-jan-2015.csv', 'sales-feb-2015.csv']
In [5]: dataframes = []
In [6]: for f in filenames:
...: dataframes.append(pd.read_csv(f))
```

- **Using a comprehension :**

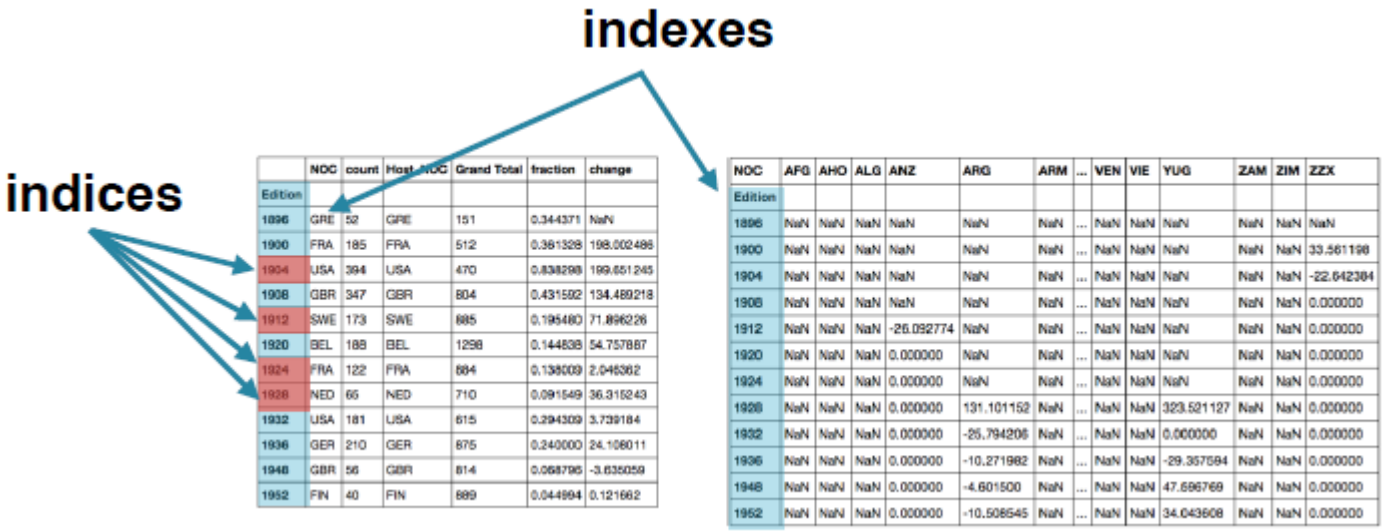
```
In [7]: filenames = ['sales-jan-2015.csv', 'sales-feb-2015.csv']
In [8]: dataframes = [pd.read_csv(f) for f in filenames]
```

- **Using glob :**

```
In [9]: from glob import glob
In [10]: filenames = glob('sales*.csv')
In [11]: dataframes = [pd.read_csv(f) for f in filenames]
```

“Indexes” vs. “Indices”

- indices: many index labels within Index data structures
- indexes: many pandas Index data structures



	NOC	count	Host_NOC	Grand Total	fraction	change
Edition						
1896	GRE	52	GRE	151	0.344371	NaN
1900	FRA	185	FRA	512	0.361328	198.002486
1904	USA	394	USA	470	0.838298	199.651245
1908	GBR	347	GBR	804	0.431592	134.489218
1912	SWE	173	SWE	885	0.195480	71.896226
1920	BEL	188	BEL	1298	0.144838	54.757887
1924	FRA	122	FRA	884	0.138009	2.046362
1928	NED	66	NED	710	0.091549	36.315243
1932	USA	181	USA	615	0.294309	3.739184
1936	GER	210	GER	875	0.240000	24.106011
1948	GBR	56	GBR	814	0.068796	-3.635059
1952	FIN	40	FIN	889	0.044994	0.121662

NOC	AFG	AHO	ALG	ANZ	ARG	ARM	...	VEN	VIE	YUG	ZAM	ZIM	ZZX
Edition													
1896	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN
1900	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	33.561198
1904	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	-22.642384
1908	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	0.000000
1912	NaN	NaN	NaN	-26.092774	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	0.000000
1920	NaN	NaN	NaN	0.000000	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	0.000000
1924	NaN	NaN	NaN	0.000000	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	0.000000
1928	NaN	NaN	NaN	0.000000	131.101152	NaN	...	NaN	NaN	323.521127	NaN	NaN	0.000000
1932	NaN	NaN	NaN	0.000000	-25.794206	NaN	...	NaN	NaN	0.000000	NaN	NaN	0.000000
1936	NaN	NaN	NaN	0.000000	-10.271982	NaN	...	NaN	NaN	-29.357594	NaN	NaN	0.000000
1948	NaN	NaN	NaN	0.000000	-4.601500	NaN	...	NaN	NaN	47.696769	NaN	NaN	0.000000
1952	NaN	NaN	NaN	0.000000	-10.508545	NaN	...	NaN	NaN	34.043608	NaN	NaN	0.000000